Guia Yaniz

Guía de la Integrante A · CRM Clínica Dental

Tu parte: Panel Laravel completo + base de la SPA (login, cartera, actividades, resumen). **Tu herramienta:** Claude Code, con desarrollo guiado por especificaciones. **Tu compañera (B):** trabaja al mismo tiempo que tú en la API y el embudo de la SPA. Ella tendrá su propia guía.

Índice

0. Cómo usar esta guía
1. Instalar el entorno en Ubuntu
2. Día 1: crear el repositorio (lo haces tú)
3. Configurar Claude Code para el proyecto
4. Fase 0 · Especificación y entorno
5. Fase 1 · Esquema, semilla y cuentas (L1, L2)
6. Fase 2 · Catálogo, pacientes y equipo (L3, L4, L5)
7. Fase 3 · Base de la SPA, planes, reportes y semilla del sector
8. Fase 4 · Vistas de la SPA (F2, F4, F5)
9. Fase 5 · Verificación y entrega
10. Cambio de requisito (spec-v2)
11. Errores típicos del agente (y material para la bitácora)
12. Preparación de la defensa
13. Apéndices

---

0. Cómo usar esta guía

Símbolos

Símbolo	Significa
💻	Comando para la terminal de Ubuntu
🤖	Prompt para pegar en Claude Code
✅	Cómo comprobar que quedó bien antes de seguir
🔄	**Punto de sincronización con B.** Avísale o espera su aviso
📝	Commit y Pull Request

El ciclo de cada feature (siempre igual, con fernando-skills)

Cada feature tiene su spec en `specs/NN-slug.md`. **El número y el nombre ya están asignados** en `specs/README.md` (sección 3.5), para que B y tú no choquen al crear specs al mismo tiempo.

```
1. Actualizar main          → git switch main && git pull
2. Crear la rama del spec   → git switch -c spec-NN-slug
(nombre de specs/README.md)
3. Abrir Claude Code        → claude   (desde la raíz del repo)
4. Diseñar el spec          → /spec <prompt 🅐 de la guía>
                              responde sus preguntas; al final guarda
specs/NN-slug.md con Estado: Borrador
5. Aprobarlo TÚ             → léelo en VS Code (fuera del chat). Si
está bien, cambia a mano
                              "**Estado:** Borrador" por "**Estado:**
Aprobado"
6. Guardar el spec          → git add specs && git commit -m "docs:
spec NN aprobado"
7. Implementar              → /spec-impl NN-slug
                              ve que ya estás en su rama y empieza por
el paso 1; se detiene después de cada paso
8. En cada paso             → git diff, pruébalo, commit del paso, y
le dices "sigue"
9. Comprobar ✅              → los pasos de la guía, o /verifica
<código>; cambia el Estado a Implementado
10. Entender el código      → /explica   (para la defensa)
```

```
11. Push y PR 📝            → B revisa el PR
12. /clear                 → antes del siguiente spec
```

> Los prompts de esta guía dan el **objetivo** y las **decisiones cerradas**: así `/spec` solo te pregunta lo que de verdad falta. Si te hace una pregunta cuya respuesta ya está en la especificación, contéstale "está en docs/especificacion.md, sección X".

Las 5 reglas para no chocar con B

1. **Solo tocas tus archivos:** `laravel/`, `frontend/` (menos `embudo.js` y `plan.js`), `docs/plan.md`, `CLAUDE.md`, `.claude/` y tus specs de `specs/`. Claude Code tendrá prohibido editar los de B (sección 3.3).
2. **Nunca trabajas en `main`.** Una rama por spec (`spec-NN-slug`, con los números asignados en `specs/README.md`), y se une con Pull Request que **B aprueba**.
3. **`git pull` al empezar cada sesión** y antes de crear cada rama.
4. **En los documentos compartidos** (`especificacion.md`, `verificacion.md`) escribes solo debajo de tus títulos.
5. **Si necesitas algo de B** (un cambio en la API, un dato), se lo pides por mensaje. No lo cambias tú.

---

1. Instalar el entorno en Ubuntu

1.1 PHP 8.4 y Composer

💻

```
sudo apt update
sudo apt install -y software-properties-common curl unzip git
sudo add-apt-repository -y ppa:ondrej/php
sudo apt update
sudo apt install -y php8.4 php8.4-cli php8.4-mysql php8.4-mbstring
php8.4-xml \
  php8.4-curl php8.4-zip php8.4-bcmath php8.4-intl php8.4-sqlite3
```

```
php -v   # debe decir 8.4.x

# Composer
cd /tmp
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
composer -V
```

1.2 MySQL 8.4 LTS (con Docker, la forma más rápida)

💻

```
docker run --name mysql-crm \
  -e MYSQL_ROOT_PASSWORD=root_local \
  -e MYSQL_DATABASE=crm_dental \
  -e MYSQL_USER=crm \
  -e MYSQL_PASSWORD=crm_local \
  -p 3306:3306 -d mysql:8.4

# Comprobar
docker exec -it mysql-crm mysql -ucrm -pcrm_local -e "SELECT VERSION();"
```

Los días siguientes solo necesitas `docker start mysql-crm` . Para entrar a la consola de MySQL:

💻

```
docker exec -it mysql-crm mysql -ucrm -pcrm_local crm_dental
```

> Si prefieres instalar MySQL sin Docker, usa el repositorio oficial (`mysql-apt-config` , eligiendo **mysql-8.4-lts**), porque el MySQL que trae Ubuntu es el 8.0.

1.3 Node.js 24 (para compilar los estilos de Breeze)

💻

```
curl -o- https://raw.githubusercontent.com/nvm-
sh/nvm/v0.40.3/install.sh | bash
source ~/.bashrc
nvm install 24
node -v   # v24.x
```

1.4 Claude Code, VS Code y utilidades

💻

```
curl -fsSL https://claude.ai/install.sh | bash   # o: npm install -g
@anthropic-ai/claude-code
claude --version
sudo apt install -y poppler-utils   # para pdftotext
```

En VS Code instala la extensión **Live Server** (sirve la SPA en `http://127.0.0.1:5500`).

1.5 fernando-skills (`/spec` y `/spec-impl`)

Si ya las tienes instaladas de la Weather App, solo comprueba que aparezcan. Si no:

💻

```
npx skills@latest add Klerith/fernando-skills
```

Si te pregunta, elige el agente **Claude Code** y el alcance **global** (para tu usuario, no para el proyecto): así no se agregan archivos al repositorio. Si el comando falla, instálalas a mano:

💻

```
git clone https://github.com/Klerith/fernando-skills ~/.fernando-skills
mkdir -p ~/.claude/skills
cp -r ~/.fernando-skills/skills/engineering/spec ~/.claude/skills/
cp -r ~/.fernando-skills/skills/engineering/spec-impl ~/.claude/skills/
```

✅ `php -v`, `composer -V`, `node -v`, `claude --version` y la consulta de MySQL funcionan, y dentro de `claude`, al escribir `/spec`, aparecen `/spec` y `/spec-impl`.

---

2. Día 1: crear el repositorio

Tú creas el repositorio porque eres la dueña de `.gitignore`, `CLAUDE.md` y `.claude/`. **B espera tu aviso para clonarlo.**

2.1 En GitHub

1. Crea el repositorio **`crm-dental`** (privado está bien), sin README.
2. *Settings* → *Collaborators* → agrega a B. Más adelante, al catedrático.
3. *Settings* → *Branches* (opcional pero recomendado): protege `main` pidiendo Pull Request con 1 aprobación.

2.2 En tu computadora

💻

```
mkdir -p ~/proyectos/crm-dental && cd ~/proyectos/crm-dental
git init -b main
mkdir -p docs .claude/commands
```

Copia ahí:

- `reparto.md` → `docs/reparto.md`
- **Esta guía no va en el repositorio** (tiene la contraseña de las semillas dentro de los prompts). Guárdala fuera, por ejemplo en `~/proyectos/guia-a.md`.
- el PDF del enunciado → `docs/enunciado.pdf`, y conviértelo a texto para que el agente lo lea fácil:

💻

```
pdftotext -layout docs/enunciado.pdf docs/enunciado.txt
```

> El enunciado queda fuera de Git (está en `.gitignore`). B hace su propia copia.

2.3 `.gitignore` de la raíz

Crea `.gitignore` con este contenido:

```
# Secretos (S2)
.env
*.env.local

# Dependencias
node_modules/
laravel/vendor/

# Compilados de Laravel
laravel/public/build/
laravel/public/hot
laravel/storage/*.key
laravel/.phpunit.result.cache

# Claude Code: ajustes personales de cada integrante
.claude/settings.local.json

# Copia local del enunciado
docs/enunciado.*
```

```
# Editores y sistema
.vscode/
.idea/
.DS_Store
```

Después agregas `CLAUDE.md`, `.claude/` y `specs/` (sección 3) y los esqueletos de `docs/` (sección 4) y haces el **único commit directo a `main`** del proyecto:

💻

```
git add .
git commit -m "inicio: estructura, reglas del agente y esqueleto de la especificación"
git remote add origin git@github.com:<tu-usuario>/crm-dental.git
git push -u origin main
```

🔄 **Aviso a B:** "Ya está el repo en GitHub. Clónalo, copia tu enunciado a `docs/` y crea tu `.claude/settings.local.json` (viene en tu guía)."

---

3. Configurar Claude Code

3.1 `CLAUDE.md` (en la raíz, lo comparten las dos)

Este archivo lo lee el agente al iniciar cada sesión. Es el entregable **P4**. Créalo con este contenido:

```
# CLAUDE.md · CRM Clínica Dental

CRM de ventas para la "Clínica Dental Sonrisa" (Quetzaltenango).
Proyecto final de Programación Web, UMES 2026.
Lo trabajan dos integrantes al mismo tiempo. Al empezar, la persona
te dice si es **A** o **B**; si no lo dice, pregunta.
```

```
## Fuentes de verdad (en este orden)
1. `docs/enunciado.txt`: el enunciado del catedrático. Manda sobre todo.
2. `docs/especificacion.md`: lo que decidimos. No se programa nada que no esté ahí.
3. `docs/plan.md`: arquitectura, contrato de la SPA, fases y tareas.
4. `docs/reparto.md`: quién es dueño de qué y acuerdos técnicos.

Si el código que te piden contradice la especificación, **detente y avisa**. No decidas por tu cuenta.

## Piezas
| Carpeta | Qué es | Tecnología | Dueña |
|---|---|---|---|
| `laravel/` | Panel de administrador y coordinador | Laravel 13, Breeze (Blade), Eloquent | A |
| `api/` | API REST del asesor | Node 24, Express 5, mysql2, jsonwebtoken, bcryptjs, cors | B |
| `frontend/` | SPA del asesor | HTML, Bootstrap 5.3, JavaScript con módulos, sin framework | A (base), B (`js/vistas/embudo.js`, `js/vistas/plan.js`) |
| `docs/` | Especificación, plan, verificación, Postman | Markdown | compartido (ver reparto) |

**Nunca edites un archivo cuya dueña es la otra integrante.** Si hace falta, dilo y para.

## Reglas que no se negocian
- **Un solo dueño del esquema:** las tablas se crean y cambian solo con migraciones de Laravel. La API nunca ejecuta CREATE ni ALTER.
**Una migración que ya está en `main` no se edita**: los cambios llegan con migraciones nuevas. La migración original de `users` de Laravel no se toca.
- **Un solo creador de cuentas:** el administrador, desde el panel. No hay registro público. Solo Laravel crea hashes (bcrypt); la API solo los verifica con `bcryptjs`.
- **Panel con Eloquent** (o el constructor de consultas).
```

`DB::raw`/`whereRaw` solo con expresiones fijas, y los datos del usuario siempre como parámetros `?`.
- **API con SQL a mano** con mysql2 y parámetros `?`. Prohibido Sequelize, Prisma, TypeORM, Knex o cualquier ORM o constructor de consultas.
- **SPA sin framework.** LocalStorage solo guarda la sesión (`token` y `usuario`).
- **Nombres:** en el código se usan los nombres del núcleo (`clientes`, `productos`, `oportunidades`, `actividades`, `vendedor`, `supervisor`). En pantalla, el vocabulario dental: Paciente, Tratamiento, Plan de tratamiento, Asesor, Coordinador, Cita. Las rutas y campos del contrato no cambian.
- **Todo en español**, con mensajes que tratan de **usted**.

Reglas técnicas compartidas (panel y API deben dar el mismo resultado)
- **Fechas:** se guardan en UTC. Laravel `APP_TIMEZONE=UTC` y `'timezone' => '+00:00'` en la conexión mysql. API: mysql2 con `timezone: 'Z'` y `SET time_zone = '+00:00'` al abrir cada conexión. Se muestran en hora de Guatemala (UTC−6 fijo).
- **Mes de una venta:** `DATE_FORMAT(fecha_cierre - INTERVAL 6 HOUR, '%Y-%m')`.
- **Dinero:** DECIMAL, nunca FLOAT. El total de una oportunidad es `SUM(cantidad * precio_unitario)` en SQL. El precio de la línea se copia del catálogo al agregar. Probabilidades: prospecto 0.10, calificado 0.25, propuesta 0.50, negociacion 0.75. Pronóstico = `ROUND(SUM(total × probabilidad), 2)` de las abiertas, sin filtro de mes.
- **Cartera máxima 10.** Para crear, tomar o asignar: transacción + `SELECT id FROM users WHERE id = ? FOR UPDATE` del vendedor, contar, asignar.
- **Orden de candados:** primero `users`, luego `clientes`, luego `oportunidades`.
- **Lo ajeno responde 404** (no 403), en las dos puertas.
- **Duplicados:** se valida antes, pero el árbitro final es el índice único de la base (error 1062 → 409 en la API, error junto al campo en el panel).

- **NIT y teléfono:** se normalizan igual en PHP y JS. Casos obligatorios en la sección 10 de la especificación.
- **XSS:** en Blade nunca `{!! !!}` con datos del usuario. En la SPA todo texto de la API pasa por `escapar()` antes de `innerHTML`.

Regla propia del sector
Un plan que incluye un tratamiento con `productos.requiere_evaluacion = true` no puede pasar a `propuesta`, `negociacion` ni `ganada`, ni recibir ese tratamiento estando en `propuesta` o `negociacion`, si el paciente no tiene al menos una actividad `reunion` completada. Responde 409. Detalle en la sección 2 de la especificación.

Semillas y cifras de control
- `php artisan migrate:fresh --seed` → semilla de verificación + semilla del sector.
- `php artisan migrate:fresh --seeder=VerificacionSeeder` → solo la de verificación (la de la defensa).
- Todas las cuentas de las dos semillas usan la contraseña pública del enunciado. **Esa contraseña solo puede escribirse en los seeders, el README y la colección de Postman**: nunca la pongas en este archivo, en la especificación, en el código del panel o de la API, ni en ningún otro lugar.
- Con la semilla de verificación: Valeria, octubre 2026 → 2 ventas, Q9,250.00, pronóstico Q2,512.88, cartera 3 de 10. Toda la empresa, octubre 2026 → 4 ventas, Q22,450.00, pronóstico Q6,751.13.

Specs por feature (fernando-skills: /spec y /spec-impl)
- `docs/especificacion.md` es la especificación general (qué hace el sistema). Cada feature tiene además su spec en `specs/NN-slug.md` (cómo se construye): se diseña con `/spec` y se implementa con `/spec-impl`.
- **El número y el nombre de cada spec ya están asignados en `specs/README.md`.** Usa siempre ese número y ese nombre, aunque veas otro número libre: las dos integrantes crean specs al mismo tiempo.
- Los specs se escriben en español: `**Estado:**`, `**Depende de:**`, `**Fecha:**`, `**Objetivo:**` y las secciones Alcance, Modelo de datos, Plan de implementación, Criterios de aceptación, Decisiones

```
tomadas y descartadas y Riesgos. Estados: Borrador, En revisión,
Aprobado, Implementado, Obsoleto.
- Cada spec dice qué requisitos de `docs/especificacion.md` cumple
(L, A, F, S, R) y no puede contradecirla. Sus criterios de aceptación
salen de los Dado/Cuando/Entonces de esos requisitos.
- El agente nunca cambia el estado de un spec: lo cambia la persona.
- Rama de cada spec: `spec-NN-slug`. La persona la crea antes de
`/spec`; `/spec-impl` la reutiliza.

## Cómo trabajar
- Antes de escribir código, presenta un plan corto (archivos y cómo
cumples cada criterio) y espera aprobación.
- No hagas `git commit` ni `git push`: los hace la persona.
- No leas ni muestres archivos `.env`; usa `.env.example`.
- Al terminar una tarea, explica cómo comprobar cada criterio de
aceptación.

## Comandos útiles
- Panel: `cd laravel && php artisan serve` (http://localhost:8000) y
`npm run dev`
- Pruebas del panel: `cd laravel && php artisan test`
- API: `cd api && npm run dev` (http://localhost:3000/api)
- SPA: Live Server sobre `frontend/` (http://127.0.0.1:5500)
```

3.2 `.claude/settings.json` (compartido, va en Git)

```
{
  "permissions": {
    "allow": [
      "Bash(php artisan *)",
      "Bash(composer *)",
      "Bash(npm install *)",
      "Bash(npm run *)",
      "Bash(node *)",
      "Bash(git status)",
      "Bash(git diff)",
```

```
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git fetch *)"
    ],
    "ask": [
      "Bash(php artisan migrate:fresh *)",
      "Bash(git commit *)",
      "Bash(git push *)"
    ],
    "deny": [
      "Read(**/.env)",
      "Edit(**/.env)",

"Edit(/laravel/database/migrations/0001_01_01_000000_create_users_tab
le.php)"
    ]
  }
}
```

> Así el agente no lee tus secretos ni edita la migración original de `users`. Tus `.env` los editas tú a mano.

3.3 `.claude/settings.local.json` (solo tuyo, no va en Git)

Este archivo es el que evita que **tu** agente toque los archivos de B. Créalo:

```
{
  "permissions": {
    "deny": [
      "Edit(/api/**)",
      "Edit(/frontend/js/vistas/embudo.js)",
      "Edit(/frontend/js/vistas/plan.js)",
      "Edit(/README.md)",
      "Edit(/docs/api.postman_collection.json)"
    ]
```

```
  }
}
```

✅ Abre `claude` en la raíz y escribe `/permissions` : deben aparecer las reglas de los dos archivos.

3.4 Comandos propios (`.claude/commands/`)

`/spec` y `/spec-impl` vienen de fernando-skills (sección 1.5) y sirven para **cada feature**. Estos 5 comandos propios cubren lo demás: redactar secciones de la especificación general, verificar, explicar para la defensa, revisar el PR de B y la bitácora. Van en Git y B los usa igual.

> **No crees un comando llamado `spec`** : chocaría con el de fernando-skills. Por eso el de las secciones se llama `/seccion` .

`.claude/commands/seccion.md`

```
---
description: Redacta una sección de docs/especificacion.md
argument-hint: <número y nombre de la sección>
---
Redacta la sección $ARGUMENTS de docs/especificacion.md.

Fuentes, en este orden:
1. docs/enunciado.txt (manda sobre todo lo demás)
2. docs/reparto.md (decisiones ya acordadas por la pareja)
3. Lo que ya está escrito en docs/especificacion.md (no lo
contradigas)

Reglas:
- Escribe SOLO dentro de la sección $ARGUMENTS. No toques otras
secciones ni los comentarios <!-- ... -->.
- Español claro y verificable. Cada requisito con criterios "Dado /
Cuando / Entonces" cuyo resultado se pueda observar (código HTTP,
mensaje, dato en pantalla o en la base).
```

```
- Nombres del núcleo para tablas, campos y rutas; vocabulario dental solo en textos de pantalla.
- No inventes reglas que choquen con el enunciado. Si algo es ambiguo, anótalo al final como "Pregunta abierta:" en vez de decidirlo.
- Al terminar, dame la lista de lo que decidiste tú (lo que no venía del enunciado ni del reparto) para revisarlo.
```

.claude/commands/verifica.md

```
---
description: Comprueba un requisito contra el código, sin modificar nada
argument-hint: <código del requisito>
---
Comprueba el requisito $ARGUMENTS contra el código actual. No modifiques archivos.

Para cada criterio Dado/Cuando/Entonces de $ARGUMENTS en docs/especificacion.md:
- di si se cumple y en qué archivo y línea,
- y cómo demostrarlo (comando, consulta SQL, pasos en el navegador o petición).

Si alguno no se cumple, dímelo sin corregirlo.
Al final, dame las filas para docs/verificacion.md con el formato:
| Criterio | Evidencia | Resultado |
```

.claude/commands/explica.md

```
---
description: Explica el código para la defensa, sin modificar nada
argument-hint: [archivo o carpeta; vacío = cambios de la rama actual]
---
Explícame, como si el catedrático me fuera a preguntar en la defensa, el código de: $ARGUMENTS
```

```
Si no hay argumento, usa los cambios de la rama actual frente a main
(git diff main...HEAD).

1. Qué hace cada archivo, en pocas líneas.
2. Qué regla del enunciado o de la especificación cumple cada parte
(R1–R8, L, A, F, S).
3. Las 5 preguntas más probables del catedrático, con una respuesta
corta.
4. Un cambio en vivo que me podría pedir, y en qué archivo y línea lo
haría.

No modifiques ningún archivo.
```

.claude/commands/revisa.md

```
---
description: Revisa el Pull Request de la compañera y explica su
código
argument-hint: <nombre de la rama>
---
Revisa los cambios de la rama $ARGUMENTS frente a main: ejecuta git
fetch y git diff main...origin/$ARGUMENTS.
Es trabajo de mi compañera. No modifiques archivos.

1. Busca: contradicciones con CLAUDE.md o con la especificación, SQL
concatenado con datos del usuario, datos expuestos (hashes,
vendedor_id, supervisor_id), problemas con peticiones simultáneas,
archivos que no son de su dueña, y textos sin escapar.
2. Dame una lista de observaciones para dejar en el PR (archivo,
línea, qué cambiar).
3. Explícame lo más importante de ese código para que yo pueda
defenderlo.
```

.claude/commands/bitacora.md

```
---
description: Redacta un caso de la bitácora de IA
argument-hint: <A o B>
---
Ayúdame a registrar un caso en la bitácora de IA de
docs/verificacion.md, en la subsección de la integrante $ARGUMENTS.
Escribe solo ahí.
Pregúntame lo que falte. Formato:

### Caso N · <título corto>
- **Requisito:** <código>
- **Qué propuso el agente:** <descripción, con un fragmento corto de código si ayuda>
- **Por qué estaba mal:** <qué regla del enunciado o de la especificación violaba>
- **Cómo lo detectamos:** <revisión, prueba, cifra de control que no cuadró…>
- **Corrección:** <qué se cambió> · commit `<hash corto>`
```

3.5 Carpeta `specs/` (va en el commit inicial)

Créala tú con dos archivos. Así los números de los specs quedan repartidos desde el día 1.

specs/README.md

```
# Specs por feature

Cada feature se diseña con `/spec` y se implementa con `/spec-impl`
(fernando-skills).
`docs/especificacion.md` dice QUÉ hace el sistema; cada spec dice
CÓMO se construye una parte.
Los números ya están asignados para que A y B no choquen al crear
specs al mismo tiempo.

| Spec | Requisitos | Dueña | Fase |
```

```
|---|---|---|---|
| 01-esquema-y-semilla | L1 | A | 1 |
| 02-api-base-login | A1, A2, base de A7 | B | 1 |
| 03-cuentas-roles | L2 | A | 1 |
| 04-catalogo | L3 | A | 2 |
| 05-pacientes | L4 (parte 1) | A | 2 |
| 06-api-clientes | A3 | B | 2 |
| 07-asignacion | L4 (parte 2) | A | 2 |
| 08-api-actividades | A5 | B | 2 |
| 09-equipo | L5 | A | 2 |
| 10-spa-base | F6, F1 | A | 3 |
| 11-api-oportunidades | A4, GET /productos | B | 3 |
| 12-planes | L6 (parte 1) | A | 3 |
| 13-api-regla-propia | Regla propia | B | 3 |
| 14-reporte | L6 (parte 2) | A | 3 |
| 15-api-resumen | A6 | B | 3 |
| 16-validacion | L7 | A | 3 |
| 17-semilla-sector | L1 (capa de sector) | A | 3 |
| 18-spa-cartera | F2 | A | 4 |
| 19-spa-embudo | F3 (tablero) | B | 4 |
| 20-spa-actividades | F4 | A | 4 |
| 21-spa-plan | F3 (detalle del plan) | B | 4 |
| 22-spa-resumen | F5 | A | 4 |
| 23-api-cierre | A7, A8, S8 | B | 5 |

Para el cambio de requisito (spec-v2) y los retos se usan números
desde el 24. Antes de usar uno, anótalo en esta tabla en un PR
pequeño.

Estados: Borrador → En revisión → Aprobado → Implementado (u
Obsoleto). Solo una persona cambia el estado.
Rama de cada spec: `spec-NN-slug`.
```

specs/.spec-config.yml

```
# spec workflow configuration
#
# AutoCreateBranch — controls whether /spec-impl creates the git
branch automatically.
#   true  (default) → /spec-impl creates and switches to spec-NN-slug
without asking
#   false           → /spec-impl asks for [y/N] confirmation before
creating the branch
AutoCreateBranch: true
```

---

4. Fase 0 · Especificación y entorno

> **Regla del enunciado:** el código llega **después** de la etiqueta `spec-v1`. En esta fase solo se escribe la especificación y se prepara el entorno (eso sí está permitido: crear el proyecto Laravel con Breeze y conectar MySQL).

Qué hace B al mismo tiempo: escribe sus secciones (contrato de la API, decisiones, requisitos A y F3) y prepara su entorno.

4.1 Esqueleto de `docs/especificacion.md` (va en el commit inicial)

Lo dejas tú con todas las secciones marcadas. Así B y tú escriben en partes distintas del archivo y Git une los cambios sin conflictos.

```
# Especificación · CRM Clínica Dental Sonrisa

> Versión: spec-v1 · Integrantes: A (nombre) y B (nombre)
> Cada sección indica su responsable. Solo se escribe debajo de los
títulos propios; las observaciones a la sección del otro van en el
Pull Request.

## 1. Sector, empresa, roles y vocabulario
```

```
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 1 -->

## 2. Regla propia del sector
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 2 -->

## 3. Modelo de datos y dónde se cumple cada regla
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 3 -->

## 4. Contrato de la API
<!-- Responsable: B -->
_Pendiente._
<!-- fin sección 4 -->

## 5. Decisiones del enunciado (sección 7)
<!-- Responsable: B -->
_Pendiente._
<!-- fin sección 5 -->

## 6. Requisitos del panel (L1–L7)
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 6 -->

## 7. Requisitos de la API (A1–A8)
<!-- Responsable: B -->
_Pendiente._
<!-- fin sección 7 -->

## 8. Requisitos de la SPA (F1–F6)

### F1 · Sesión
```

```
<!-- Responsable: A -->
_Pendiente._

### F2 · Cartera
<!-- Responsable: A -->
_Pendiente._

### F3 · Embudo
<!-- Responsable: B -->
_Pendiente._

### F4 · Actividades
<!-- Responsable: A -->
_Pendiente._

### F5 · Resumen del mes
<!-- Responsable: A -->
_Pendiente._

### F6 · Calidad de la interfaz
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 8 -->

## 9. Seguridad (S1–S8)
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 9 -->

## 10. Casos compartidos de NIT y teléfono
<!-- Acordado por A y B. Panel y API deben dar exactamente este
resultado -->

| Entrada | Se guarda como | ¿Válido? |
|---|---|---|
| `452 1817-k` | `4521817K` | sí |
| `4521817-K` | `4521817K` | sí (repetido del anterior → 409) |
```

```
| `00012345679` | `12345679` | sí |
| `cf` / `CF` | `NULL` (la API devuelve `"CF"`) | sí |
| `4521817-1` | — | no (verificador incorrecto) |
| `+502 5555-1234` | `55551234` | sí |
| `(502) 7761.2345` | `77612345` | sí |
| `50255551234` | `55551234` | sí |
| `5021 2345` | `50212345` | sí (no se quita el 502) |
| `1234-5678` | — | no (empieza con 1) |
| `5555123` | — | no (7 dígitos) |

Regla del prefijo: `+502` se quita siempre; `502` sin `+` solo si después quedan exactamente 8 dígitos.
<!-- fin sección 10 -->

## 11. Cifras de control
<!-- Responsable: A -->
_Pendiente._
<!-- fin sección 11 -->
```

4.2 Esqueleto de `docs/plan.md` (va en el commit inicial)

Pon solo los títulos por ahora; lo llenas en el paso 4.5.

```
# Plan · CRM Clínica Dental Sonrisa

## 1. Arquitectura
## 2. Estructura del repositorio
## 3. Contrato de la SPA
## 4. Fases y tareas
## 5. Qué se le pide al agente en cada fase
## 6. Historial de cambios del plan
```

4.3 Escribir tus secciones de la especificación

Haz cada una en su propia rama corta, para que B la revise.

💻

```
git switch main && git pull
git switch -c docs/spec-a
claude
```

Secciones 1 y 2: sector y regla propia

🤖

```
Soy la integrante A. Lee docs/enunciado.txt (secciones 3, 5 y 7) y
docs/reparto.md completo.

Redacta las secciones 1 y 2 de docs/especificacion.md, con las mismas
reglas del comando /seccion (solo dentro de esas secciones,
Dado/Cuando/Entonces, y al final la lista de lo que decidiste tú).

Sección 1 · Sector, empresa, roles y vocabulario:
- Empresa ficticia: "Clínica Dental Sonrisa", en Quetzaltenango. Qué
vende: tratamientos dentales. Quién usa cada puerta.
- Tabla de los tres roles (administrador, coordinador = supervisor,
asesor = vendedor): puerta, qué puede hacer y qué no (según la
sección 5 del enunciado).
- Tabla de vocabulario núcleo → pantalla (copia la del reparto,
incluidas las etapas y los tipos de actividad).
- Aclara que las rutas, tablas y campos del contrato NO cambian: solo
los textos.

Sección 2 · Regla propia:
- Enunciado de la regla exactamente como está en el reparto.
- Por qué tiene sentido en una clínica (ortodoncia e implantes
necesitan evaluación clínica y radiografía antes de presupuestar).
- Dónde se cumple: columna productos.requiere_evaluacion (booleano,
por omisión falso, migración nueva); el administrador la marca en el
catálogo del panel; la API la verifica en PATCH
/oportunidades/:id/etapa (al pasar a propuesta, negociacion o ganada)
```

```
y en POST /oportunidades/:id/productos (si el plan ya está en
propuesta o negociacion). Respuesta 409 con { error } y el mensaje
del reparto.
- Qué cuenta como evaluación: una actividad tipo reunion, completada,
de ese mismo paciente (de cualquier asesor).
- Por qué no choca con el núcleo: con solo la semilla de verificación
ningún producto requiere evaluación, así que la regla nunca se
activa.
- Al menos 5 criterios Dado/Cuando/Entonces: pasa a propuesta sin
cita → 409 y sin registro en historial; con cita completada → 200;
cita pendiente (no completada) no cuenta; agregar el tratamiento a un
plan en negociacion sin cita → 409; plan sin tratamientos que
requieran evaluación avanza normal.
- Cómo se prueba: peticiones de Postman (las hace B) con datos de la
semilla del sector.
```

Sección 3: modelo de datos

🤖

```
/seccion 3. Modelo de datos y dónde se cumple cada regla

Contexto extra: usa la sección 6 del enunciado. Para cada tabla
(users, productos, clientes, oportunidades, oportunidad_productos,
historial_etapas, actividades) haz una tabla con: columna, tipo
MySQL, reglas, y "dónde se cumple" (BD: FK/UNIQUE/CHECK · Panel:
FormRequest · API: validación).

Incluye estas decisiones nuestras:
- productos.requiere_evaluacion BOOLEAN NOT NULL DEFAULT FALSE (capa
de sector, no rompe el contrato).
- Índice único (oportunidad_id, etapa_nueva) en historial_etapas:
como no hay retrocesos, cada etapa aparece una vez por oportunidad;
es el árbitro final ante dos cambios simultáneos.
- Las restricciones CHECK con su nombre y expresión:
  chk_users_supervisor: (rol = 'vendedor' AND supervisor_id IS NOT
```

```
NULL) OR (rol <> 'vendedor' AND supervisor_id IS NULL)
  chk_productos_precio: precio > 0
  chk_clientes_telefono: telefono REGEXP '^[2-9][0-9]{7}$'
  chk_clientes_nit: nit IS NULL OR nit REGEXP '^[1-9][0-9]*[0-9K]$'
  chk_oportunidades_motivo: (etapa = 'perdida') = (motivo_perdida IS
NOT NULL)
  chk_oportunidades_motivo_vacio: motivo_perdida IS NULL OR
CHAR_LENGTH(TRIM(motivo_perdida)) BETWEEN 1 AND 255
  chk_oportunidades_cierre: (etapa IN ('ganada','perdida')) =
(fecha_cierre IS NOT NULL)
  chk_lineas_cantidad: cantidad BETWEEN 1 AND 999
  chk_lineas_precio: precio_unitario > 0
- Llaves foráneas: cascada al borrar en
oportunidad_productos.oportunidad_id, historial_etapas.oportunidad_id
y actividades.cliente_id. Las demás sin acción (restringen). Explica
que users.supervisor_id NO puede tener acción ON DELETE porque MySQL
prohíbe acciones referenciales en columnas usadas por un CHECK (y
además las cuentas no se borran).
- Lo que no puede vivir en la base y por qué: "supervisor_id debe ser
un supervisor activo" y "máximo 10 clientes" (involucran otras
filas), y cómo se cumplen en panel y API (validación + candado FOR
UPDATE).
- Todas las fechas en UTC.
```

Sección 6: requisitos del panel

🤖

```
/seccion 6. Requisitos del panel (L1–L7)

Reescribe L1 a L7 del enunciado con criterios Dado/Cuando/Entonces
verificables. Además de todo lo que pide el enunciado, incluye estas
decisiones nuestras:
- L1: migraciones con los nombres extender_users, crear_productos,
crear_clientes, crear_oportunidades, crear_oportunidad_productos,
crear_historial_etapas, crear_actividades y
```

```
agregar_requiere_evaluacion. Criterios: migrate:fresh --seed y
migrate:fresh --seeder=VerificacionSeeder sin pasos manuales; conteos
esperados de la semilla (7 cuentas, 5 productos, 21 clientes, 11
oportunidades, 13 líneas, 43 registros de historial, 4 actividades);
un INSERT con teléfono inválido falla por CHECK.
- L2: sin página ni ruta de registro, sin "olvidé mi contraseña" y
sin "eliminar cuenta" en el perfil. El asesor que intenta entrar ve:
"Su cuenta es de asesor. Ingrese desde la aplicación del asesor." Una
cuenta desactivada ve: "Su cuenta está desactivada. Comuníquese con
el administrador." El administrador no puede desactivarse ni
cambiarse el rol. Un coordinador que tiene asesores no se puede
desactivar ni cambiar de rol hasta reasignarlos.
- L3: tratamientos con casilla "Requiere evaluación".
- L4: decisiones de la sección 7 del reparto: al reasignar, los
planes abiertos pasan al nuevo asesor y los cerrados se quedan con el
anterior; no se libera un paciente con planes abiertos (mensaje: "No
se puede liberar: el paciente tiene planes de tratamiento
abiertos."); los pacientes de un asesor desactivado se quedan
asignados. Cartera llena: "La cartera de <nombre> ya tiene 10
pacientes, el máximo permitido."
- L5: página "Mi equipo" con cada asesor: estado, cartera x de 10,
ventas y total del mes actual de Guatemala.
- L6: planes de solo lectura con filtros etapa y asesor, total y
fecha de cierre en hora de Guatemala; reporte con selector de mes
(por omisión el mes actual de Guatemala) por asesor, por tratamiento
y, para el administrador, por equipo. Criterio: con la semilla de
verificación coincide con las cifras de control.
- L7: validación con FormRequest, mensajes en español tratando de
usted, old() y error junto a cada campo; un 1062 de la base también
se muestra junto al campo.
```

Sección 8: tus requisitos de la SPA (F1, F2, F4, F5, F6)

🤖

```
/seccion 8. Requisitos de la SPA, solo F1, F2, F4, F5 y F6 (no toques F3, es de B)

Usa F1–F6 del enunciado y estas decisiones:
- F1: vista de inicio de sesión; la sesión ({ token, usuario }) se guarda en localStorage con la clave "sesion" y es lo único que se guarda ahí. 403 → se muestra el mensaje de la API. 401 durante la sesión → se borra la sesión y se vuelve al login con el aviso "Su sesión terminó. Ingrese de nuevo."
- F2: lista de pacientes con búsqueda en el servidor (?q=), contador "x de 10" (de GET /reportes/resumen), alta de paciente con errores del 422 junto a cada campo y lo escrito conservado, lista de disponibles con botón "Tomar" y el mensaje del 409.
- F4: pestañas Pendientes y Completadas (?estado=), las vencidas resaltadas en rojo con la etiqueta "Vencida", nueva actividad (paciente, tipo Llamada/Cita/Tarea, descripción, fecha y hora) enviada en ISO 8601 con -06:00, botón "Marcar como completada".
- F5: selector de mes, tarjetas de citas cerradas (ventas), total vendido, pronóstico y cartera, montos en quetzales (Q9,250.00).
- F6: patrón estado → render, delegación de eventos con data-accion y data-form, indicador de carga mientras hay peticiones, mensaje "No se pudo conectar con el servidor. Verifique que la API esté encendida." si la API está apagada, todo texto escapado, sin desplazamiento horizontal a 375 px.
Rutas de la SPA: #/login, #/cartera, #/embudo, #/plan/:id, #/actividades, #/resumen.
```

Secciones 9 y 11: seguridad y cifras de control

🤖

```
/seccion 9. Seguridad (S1–S8)

Para cada regla S1 a S8 del enunciado: cómo la cumplimos (en qué pieza y con qué mecanismo concreto de nuestro proyecto) y cómo se comprueba (una prueba concreta). Incluye las pruebas del enunciado:
```

```
Valeria intenta leer y modificar un paciente de Diego (404); Sergio
intenta editar un paciente del equipo de Sandra (404); un paciente
llamado <img src=x onerror=alert(1)> se muestra como texto en el
panel y en la SPA; diez tomas simultáneas del mismo paciente (solo
una gana).
```

🤖

```
/seccion 11. Cifras de control

Copia las dos tablas de cifras de control de la sección 11 del
enunciado (por persona/equipo/empresa y por producto de octubre 2026)
y agrega los tres "Si sus cifras no coinciden" como lista de errores
que vamos a vigilar.
```

✅ **Revisa tú misma cada sección** antes de subirla. Lee en especial la lista de "lo que decidió el agente" que te da `/seccion`.

📝

```
git add docs/especificacion.md
git commit -m "docs: secciones de la especificación de A"
git push -u origin docs/spec-a
```

Abre el PR en GitHub (o con `gh pr create --fill`). **B lo revisa**; tú revisas el de ella con:

🤖

```
/revisa docs/spec-b
```

> En la revisión fíjate en que el contrato de B (sección 4) use los mismos nombres de campos y las mismas decisiones que tus secciones 3 y 6.

4.4 Preparar el entorno de Laravel (permitido en la fase 0)

Esto no es código de requisitos: es el esqueleto que el enunciado pide preparar mientras escriben la especificación.

💻

```
git switch main && git pull
git switch -c panel/entorno
composer create-project laravel/laravel laravel
cd laravel
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run build
```

Edita **a mano** `laravel/.env` (el agente no puede leerlo):

```
APP_NAME="Clínica Dental Sonrisa"
APP_LOCALE=es
APP_TIMEZONE=UTC
APP_URL=http://localhost:8000

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=crm_dental
DB_USERNAME=crm
DB_PASSWORD=crm_local
```

Y deja `laravel/.env.example` con las mismas claves, pero con `DB_PASSWORD=` vacío.

En `laravel/config/database.php`, dentro de `'mysql' => [ ... ]`, agrega:

```
'timezone' => '+00:00',
```

💻

```
php artisan migrate      # crea las tablas que trae Laravel, ya en
MySQL
php artisan serve        # abre http://localhost:8000
```

✅ Ves la página de Laravel, `/login` funciona, y en MySQL existen las tablas `users`, `sessions`, `cache` y `jobs`:

💻

```
docker exec -it mysql-crm mysql -ucrm -pcrm_local crm_dental -e "SHOW
TABLES;"
```

📝

```
cd ..
git add laravel .gitignore
git commit -m "panel: proyecto Laravel con Breeze conectado a MySQL"
git push -u origin panel/entorno
```

4.5 Escribir `docs/plan.md` (P2, tuyo)

Copia este contenido. Las secciones 1 a 3 son tuyas y B las necesita **tal cual** para hacer el embudo.

````
# Plan · CRM Clínica Dental Sonrisa

## 1. Arquitectura

```
Navegador (coordinador/admin) ──► Panel Laravel :8000 ──┐
                                                        ├──► MySQL
8.4 · crm_dental
````
````

```
Navegador (asesor) ──► SPA :5500 ──► API Express :3000 ──┘
```

- **Panel (monolito):** Blade arma las páginas en el servidor. Único dueño del esquema (migraciones) y único creador de cuentas.
- **API (servicio desacoplado):** JWT HS256, SQL a mano con mysql2. Lee y escribe sobre las tablas que definió Laravel.
- **SPA:** HTML + Bootstrap 5.3 + módulos de JavaScript, sin framework. Consume solo la API.

2. Estructura del repositorio

```
crm-dental/
├── README.md                       (B)
├── CLAUDE.md                       (A)
├── .claude/                        (A)
├── specs/                          specs por feature (números en specs/README.md)
├── docs/
│   ├── reparto.md
│   ├── especificacion.md           (A y B, por secciones)
│   ├── plan.md                     (A)
│   ├── verificacion.md             (B crea; A y B llenan)
│   └── api.postman_collection.json (B)
├── laravel/                        (A)
│   ├── app/Http/Controllers/       CuentaController, ProductoController, ClienteController,
│   │                               EquipoController, OportunidadController, ReporteController
│   ├── app/Http/Requests/          CuentaRequest, ProductoRequest, ClienteRequest, AsignarRequest
│   ├── app/Http/Middleware/        CuentaPanel, Rol
│   ├── app/Services/Cartera.php    asignar, reasignar y liberar con candado
│   ├── app/Support/                Nit.php, Telefono.php, Formato.php
│   ├── app/Rules/                  NitValido.php, TelefonoValido.php
```

```
|   ├── database/seeders/          VerificacionSeeder, SectorSeeder
|   └── resources/views/           cuentas/, productos/, clientes/,
equipo/, oportunidades/, reportes/
├── api/                           (B)
|   └── src/  app.js, servidor.js, db.js, middleware/, rutas/, util/
└── frontend/
    ├── index.html                 (A)
    ├── css/estilos.css            (A)
    └── js/
        ├── app.js estado.js api.js sesion.js util.js textos.js   (A)
        └── vistas/
            ├── login.js cartera.js actividades.js resumen.js     (A)
            └── embudo.js plan.js                                 (B)
```

3. Contrato de la SPA

Este contrato permite que A y B construyan vistas distintas al mismo tiempo sin tocar los mismos archivos.

3.1 Rutas (las registra A en `app.js`)

Hash	Vista	Dueña
`#/login`	`vistas/login.js`	A
`#/cartera`	`vistas/cartera.js`	A
`#/embudo`	`vistas/embudo.js`	B
`#/plan/:id`	`vistas/plan.js` (recibe `params.id`)	B
`#/actividades`	`vistas/actividades.js`	A
`#/resumen`	`vistas/resumen.js`	A

Sin sesión, cualquier ruta lleva a `#/login`. Con sesión, `#/` lleva a `#/cartera`.

3.2 Forma de una vista

Cada archivo de `vistas/` exporta por omisión un objeto así:

```js
export default {
  titulo: 'Texto para el menú y el título',
  // Se llama al entrar a la ruta. Pide datos a la API y los guarda con cambiar().
  async cargar(params) {},
  // Devuelve el HTML de la vista como texto. TODO dato de la API pasa por escapar().
  render(estado) { return ''; },
  // Clic en un elemento con data-accion="nombre". Recibe el elemento y el evento.
  acciones: {
    async nombre(elemento, evento) {}
  },
  // Envío de <form data-form="nombre">. Recibe los datos del formulario como objeto y el form.
  formularios: {
    async nombre(datos, form) {}
  }
};
```

`app.js` tiene **un solo** escuchador de `click` y uno de `submit` sobre `#app` (delegación de eventos) que busca `data-accion` o `data-form` y llama a la función de la vista activa. Las vistas no agregan escuchadores propios.

3.3 Estado (`estado.js`)

```js
estado = {
  sesion: null,        // { token, usuario: { id, nombre, correo } }
  ruta: 'cartera',     // nombre de la ruta activa
  params: {},          // ej. { id: '5' } en #/plan/5
  cargando: 0,         // peticiones en curso (> 0 muestra el indicador)
```

```
  aviso: null,          // { tipo: 'success' | 'danger' | 'warning', texto }
  datos: {},            // cada vista usa SOLO su clave: datos.cartera, datos.embudo, datos.plan…
  formularios: {}       // { [nombreForm]: { valores, errores } } para conservar lo escrito
}
```

- `cambiar(parcial)`: mezcla `parcial` en el estado y vuelve a dibujar. **Es la única forma de cambiar el estado.**
- Para los datos de una vista: `cambiar({ datos: { ...estado.datos, embudo: lista } })`.

3.4 Funciones compartidas

Archivo	Función	Qué hace
`api.js`	`pedir(metodo, ruta, cuerpo?)`	`fetch` a `http://localhost:3000/api` + ruta, con el token. Devuelve el JSON (o `null` en 204). Lleva la cuenta de `estado.cargando`. Si falla lanza `ErrorApi`
`api.js`	`class ErrorApi`	`{ status, message, errores }`. `errores` viene en los 422 (`{ campo: mensaje }`). `status` 0 = la API no responde
`api.js`	(automático)	Con 401 en plena sesión: borra la sesión, pone el aviso "Su sesión terminó. Ingrese de nuevo." y va a `#/login`
`util.js`	`escapar(texto)`	Escapa `& < > " '`. Acepta `null`/números
`util.js`	`quetzales(n)`	`9250` → `Q9,250.00`
`util.js`	`fechaGT(iso)`	Fecha y hora en Guatemala, ej. `5 oct 2026, 10:00`
`util.js`	`aISO(valorLocal)`	`'2026-12-01T10:00'` (de un `datetime-local`) → `'2026-12-01T10:00:00-06:00'`
`util.js`	`mesActualGT()`	`'2026-11'` según la hora de Guatemala

```
| `util.js` | `errorCampo(errores, campo)` | HTML del mensaje de error de ese campo (escapado), o `''` |
| `util.js` | `claseError(errores, campo)` | `' is-invalid'` o `''` |
| `util.js` | `mostrarError(e)` | Pone `e.message` como aviso de tipo `danger` |
| `textos.js` | `ETAPAS`, `ABIERTAS`, `CERRADAS`, `PROBABILIDAD`, `TIPOS` | Nombres de pantalla de etapas y tipos de actividad, probabilidades |

## 4. Fases y tareas

| Fase | Tarea | Requisito | Responsable | Estado |
|---|---|---|---|---|
| 0 | Especificación (sus secciones) | P1 | A y B | |
| 0 | Entorno Laravel + Breeze + MySQL | — | A | |
| 0 | Entorno de la API | — | B | |
| 1 | Migraciones, modelos y VerificacionSeeder | L1 | A | |
| 1 | Cuentas y roles | L2 | A | |
| 1 | Base de la API, login y middleware | A1, A2 | B | |
| 2 | Catálogo | L3 | A | |
| 2 | Pacientes y asignación | L4 | A | |
| 2 | Mi equipo | L5 | A | |
| 2 | Clientes y actividades de la API | A3, A5 | B | |
| 3 | Base de la SPA y login | F6, F1 | A | |
| 3 | Planes y reportes del panel | L6 | A | |
| 3 | Validación en español | L7 | A | |
| 3 | Semilla del sector | L1 | A | |
| 3 | Oportunidades, regla propia, resumen y productos | A4, A6 | B | |
| 4 | Cartera, actividades y resumen de la SPA | F2, F4, F5 | A | |
| 4 | Embudo y detalle del plan | F3 | B | |
| 4 | Colección de Postman y errores | A7, A8 | B | |
| 5 | Verificación, bitácora y README | P3, P5, A8 | A y B | |

## 5. Qué se le pide al agente en cada fase

- **Fase 0:** redactar secciones de la especificación general con
```

```
`/seccion`, revisando las decisiones que tomó.
- **Fases 1 a 5:** cada feature con `/spec` (el agente pregunta y escribe `specs/NN-slug.md`), aprobación humana del spec y `/spec-impl NN-slug` (implementa paso a paso con pausas). Después `/verifica <código>` y `/explica`. Los números de los specs están en `specs/README.md`. Prompts completos en las guías de A y B.
- **Revisión:** `/revisa <rama>` sobre el PR de la compañera.
- **Fase 5:** `/verifica` de cada requisito para llenar la matriz; `/bitacora` para cada caso.
- **Nunca se le pide:** hacer commits o push, leer `.env`, editar migraciones ya unidas a main, tocar archivos de la otra integrante.

## 6. Historial de cambios del plan

| Fecha | Cambio |
|---|---|
| | Versión inicial |
```

📝

```
git switch main && git pull && git switch -c docs/plan
# pega el contenido en docs/plan.md
git add docs/plan.md && git commit -m "docs: plan con arquitectura, contrato de la SPA y fases"
git push -u origin docs/plan
```

4.6 🔄 Cerrar la fase 0: etiqueta `spec-v1`

Cuando los PR de las dos especificaciones y del plan estén unidos a `main`:

💻

```
git switch main && git pull
git tag -a spec-v1 -m "Especificación versión 1"
git push origin spec-v1
```

🔄 **Aviso a B:** "Ya está `spec-v1`. Podemos empezar a programar."

> Desde aquí, **antes de cada fase**, agrega una línea en la sección 6 de `plan.md` ("Fase 2: empezamos L3–L5 y A3, A5") en el mismo PR de la primera tarea de la fase. El enunciado pide que el plan se actualice antes de cada fase.

---

5. Fase 1 · Esquema, semilla y cuentas

Qué hace B al mismo tiempo: la base de la API (conexión, errores, NIT y teléfono en JS) y el login con JWT. **Ella necesita tus migraciones y la semilla de verificación cuanto antes**: es tu primera prioridad.

5.1 Spec 01 · Esquema y semilla de verificación (L1)

💻

```
git switch main && git pull && git switch -c spec-01-esquema-y-
semilla && claude
```

🤖

```
/spec Soy la integrante A. Spec 01-esquema-y-semilla (requisito L1).
Objetivo: crear el esquema completo del núcleo en MySQL con
migraciones de Laravel y cargar la semilla de verificación.

Decisiones cerradas (parte 1, esquema y modelos). Lee CLAUDE.md, la
sección 3 y L1 de docs/especificacion.md, y la sección 6 de
docs/enunciado.txt.

Crea estas migraciones con php artisan make:migration, en este orden
(por las llaves foráneas):
1. extender_users (--table=users): rol
ENUM('administrador','supervisor','vendedor') NOT NULL DEFAULT
'vendedor', supervisor_id nullable FK a users.id SIN acciones ON
```

```
DELETE/ON UPDATE, activo BOOLEAN DEFAULT TRUE. CHECK
chk_users_supervisor.
2. crear_productos: nombre VARCHAR(120) único, precio DECIMAL(10,2),
activo BOOLEAN DEFAULT TRUE, timestamps. CHECK chk_productos_precio.
3. crear_clientes: vendedor_id nullable FK a users, nombre
VARCHAR(150), nit VARCHAR(12) nullable único, telefono CHAR(8) único,
correo VARCHAR(150) nullable, timestamps. CHECK chk_clientes_telefono
y chk_clientes_nit.
4. crear_oportunidades: cliente_id FK, vendedor_id FK, titulo
VARCHAR(150), etapa
ENUM('prospecto','calificado','propuesta','negociacion','ganada','per
dida') DEFAULT 'prospecto', motivo_perdida VARCHAR(255) nullable,
fecha_cierre DATETIME nullable (usa dateTime, no timestamp),
timestamps. CHECK chk_oportunidades_motivo,
chk_oportunidades_motivo_vacio y chk_oportunidades_cierre.
5. crear_oportunidad_productos: oportunidad_id FK cascadeOnDelete,
producto_id FK, cantidad SMALLINT UNSIGNED, precio_unitario
DECIMAL(10,2), timestamps, índice único (oportunidad_id,
producto_id). CHECK chk_lineas_cantidad y chk_lineas_precio.
6. crear_historial_etapas: oportunidad_id FK cascadeOnDelete,
usuario_id FK a users, etapa_anterior ENUM(mismas etapas) nullable,
etapa_nueva ENUM(mismas etapas), created_at DATETIME (sin
updated_at), índice único (oportunidad_id, etapa_nueva).
7. crear_actividades: cliente_id FK cascadeOnDelete, vendedor_id FK a
users, tipo ENUM('llamada','reunion','tarea'), descripcion
VARCHAR(500), vence_en DATETIME nullable, completada BOOLEAN DEFAULT
FALSE, timestamps.
8. agregar_requiere_evaluacion (--table=productos):
requiere_evaluacion BOOLEAN NOT NULL DEFAULT FALSE.

Reglas:
- Las CHECK van con DB::statement('ALTER TABLE ... ADD CONSTRAINT
<nombre> CHECK (...)') después de crear la tabla, con las expresiones
exactas de la sección 3 de la especificación, y el down() las quita.
- NO toques 0001_01_01_000000_create_users_table.php.
- "negociacion" va sin tilde en la base.
```

```
Modelos (con $table explícito en todos, porque Laravel pluraliza en
inglés):
- User: agrega rol, supervisor_id y activo a $fillable; cast activo
boolean; métodos esAdmin(), esSupervisor(), esVendedor(); relaciones
supervisor(), vendedores(), clientes(), oportunidades().
- Producto (productos), Cliente (clientes), Oportunidad
(oportunidades), OportunidadProducto (oportunidad_productos),
HistorialEtapa (historial_etapas, const UPDATED_AT = null), Actividad
(actividades).
- Casts: precio y precio_unitario 'decimal:2', booleanos,
fecha_cierre / vence_en / created_at 'datetime'.
- En Oportunidad: constantes ETAPAS, ABIERTAS y PROBABILIDAD
(prospecto 10, calificado 25, propuesta 50, negociacion 75, ganada
100, perdida 0) y relaciones cliente(), vendedor(), lineas(),
historial().

En el plan de implementación: primero las migraciones, luego los
modelos y al final la semilla (parte 2, abajo).
```

Pega este segundo bloque en el mismo mensaje de `/spec`, debajo del anterior (es la parte 2 del mismo spec):

🤖

```
Decisiones cerradas (parte 2, VerificacionSeeder). Fuente
obligatoria: la sección 11 de docs/enunciado.txt. Carga EXACTAMENTE
esos datos, ni uno más.

- Cuentas: las 7 de la tabla, con Hash::make('Ventas2026'), rol,
supervisor e Iván Toj inactivo. Correos exactos.
- Productos: los 5, con "Licencia anterior" inactivo.
requiere_evaluacion = false en todos.
- Clientes: los 21, con NIT compacto tal como está en la tabla y null
cuando dice CF. Teléfonos de 8 dígitos. Correo solo donde aparece.
- Oportunidades: las 11, con su etapa, motivo de pérdida ("Eligió a
la competencia") y fecha_cierre en UTC tal como dice la tabla.
```

```
- Líneas: cantidad × precio GUARDADO de la tabla (ojo: "Renovación de
planes" guarda Plan avanzado a 2600.00, no al precio del catálogo).
- Historial: 43 registros. Cada oportunidad tiene uno de creación
(etapa_anterior null, etapa_nueva prospecto) y uno por cada avance
hasta su etapa actual; las ganadas pasan por las 4 abiertas; "Plan
familiar" se pierde desde propuesta. usuario_id = su vendedor.
Fechas: 3 días entre registros, terminando en la fecha de cierre, o
en 2026-10-02 16:00:00 UTC si sigue abierta. El created_at de cada
oportunidad = la fecha de su registro de creación.
- Actividades: las 4, con vence_en en UTC y completada según la
tabla.

Después:
- Crea SectorSeeder vacío (con un comentario "se llena en la fase
3").
- DatabaseSeeder llama a VerificacionSeeder y luego a SectorSeeder.
Inserta los datos directamente (Eloquent o DB::table), sin pasar por
validaciones del panel.
```

➡️ Cuando `/spec` guarde `specs/01-esquema-y-semilla.md` : léelo en VS Code (revisa que el plan tenga migraciones → modelos → semilla, y que los criterios incluyan los conteos y las cifras de control), cámbialo a **Aprobado**, haz commit del spec y ejecuta:

🤖

```
/spec-impl 01-esquema-y-semilla
```

✅ Cuando termine el paso de las migraciones, comprueba:

💻

```
cd laravel && php artisan migrate:fresh
docker exec -it mysql-crm mysql -ucrm -pcrm_local crm_dental \
  -e "SELECT TABLE_NAME, CONSTRAINT_NAME FROM
```

```
information_schema.CHECK_CONSTRAINTS c JOIN
information_schema.TABLE_CONSTRAINTS t USING (CONSTRAINT_NAME) WHERE
t.TABLE_SCHEMA='crm_dental';"
```

Deben salir las 9 restricciones `chk_...` . Prueba que una falle a propósito:

```
INSERT INTO clientes (nombre, telefono, created_at, updated_at)
VALUES ('Prueba', '12345678', NOW(), NOW());
-- Debe fallar: Check constraint 'chk_clientes_telefono' is violated.
```

🤖 `/explica` → léelo con calma. En la defensa te pueden preguntar por qué las CHECK viven en la base si el panel ya valida.

📝 `git commit -m "panel: migraciones del núcleo con llaves, únicos y CHECK"`

✅ Cuando termine el paso de la semilla, comprueba:

💻

```
php artisan migrate:fresh --seeder=VerificacionSeeder
docker exec -it mysql-crm mysql -ucrm -pcrm_local crm_dental
```

```
SELECT
 (SELECT COUNT(*) FROM users) AS cuentas,                  -- 7
 (SELECT COUNT(*) FROM productos) AS productos,            -- 5
 (SELECT COUNT(*) FROM clientes) AS clientes,              -- 21
 (SELECT COUNT(*) FROM oportunidades) AS oportunidades,    -- 11
 (SELECT COUNT(*) FROM oportunidad_productos) AS lineas,   -- 13
 (SELECT COUNT(*) FROM historial_etapas) AS historial,     -- 43
 (SELECT COUNT(*) FROM actividades) AS actividades;        -- 4

-- Valeria, octubre 2026: debe dar 2 y 9250.00
SELECT COUNT(DISTINCT o.id) AS ventas, SUM(l.cantidad *
l.precio_unitario) AS total
FROM oportunidades o
```

```
  JOIN oportunidad_productos l ON l.oportunidad_id = o.id
  JOIN users u ON u.id = o.vendedor_id
  WHERE u.email = 'valeria@crm.test' AND o.etapa = 'ganada'
    AND DATE_FORMAT(o.fecha_cierre - INTERVAL 6 HOUR, '%Y-%m') = '2026-
  10';

  -- Pronóstico de toda la empresa: debe dar 6751.13
  SELECT ROUND(SUM(l.cantidad * l.precio_unitario *
    CASE o.etapa WHEN 'prospecto' THEN 0.10 WHEN 'calificado' THEN 0.25
                 WHEN 'propuesta' THEN 0.50 WHEN 'negociacion' THEN
  0.75 END), 2) AS pronostico
  FROM oportunidades o
  JOIN oportunidad_productos l ON l.oportunidad_id = o.id
  WHERE o.etapa IN
  ('prospecto','calificado','propuesta','negociacion');

  -- El hash debe empezar con $2y$
  SELECT email, LEFT(password, 4) FROM users LIMIT 3;
```

📝

```
git add . && git commit -m "panel: semilla de verificación con 43
registros de historial"
git push -u origin spec-01-esquema-y-semilla
```

🔄 **Aviso a B (importante):** "Abrí el PR `spec-01-esquema-y-semilla`. Revísalo y únelo. Luego corre en tu compu: `git pull && cd laravel && composer install && cp .env.example .env && php artisan key:generate` (pon tu contraseña de MySQL en `.env`) `&& php artisan migrate:fresh --seed`."

🔄 **Prueba de integración 1** (cuando B tenga el login): pídele que haga login con `valeria@crm.test` / `Ventas2026` contra tu base. Si responde con token, el hash `$2y$` de Laravel y `bcryptjs` se entienden. Pruébalo tú también:

💻

```
curl -s -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"correo":"valeria@crm.test","clave":"Ventas2026"}'
```

5.3 L2 · Cuentas y roles

💻

```
git switch main && git pull && git switch -c spec-03-cuentas-roles && claude
```

🤖

```
/spec Soy la integrante A. Spec 03-cuentas-roles (requisito L2).
Objetivo: que el administrador gestione las cuentas y que cada rol entre solo por su puerta.

Decisiones cerradas:
1. Quitar de Breeze: rutas, controladores y vistas de registro, de "olvidé mi contraseña"/restablecer, y la sección y ruta de "eliminar cuenta" del perfil. Quitar también los enlaces que llevan a ellas (bienvenida, login, perfil). Deja el cambio de contraseña del perfil.
2. En app/Http/Requests/Auth/LoginRequest.php: después de validar las credenciales, si la cuenta es de vendedor → cerrar sesión y error en el campo email "Su cuenta es de asesor. Ingrese desde la aplicación del asesor."; si está inactiva → "Su cuenta está desactivada. Comuníquese con el administrador."
3. Middleware CuentaPanel (alias 'panel') en todas las rutas autenticadas: si la cuenta ya no está activa o es de vendedor, cierra la sesión, invalida la sesión y redirige a /login con el mensaje. Así una cuenta desactivada pierde la sesión en su siguiente petición.
4. Middleware Rol (alias 'rol', uso rol:administrador o rol:administrador,supervisor) que responde 403 si el rol no está permitido. Registra los alias en bootstrap/app.php.
```

```
5. CuentaController (solo administrador): index (tabla con nombre, correo, rol en vocabulario de pantalla, coordinador, estado; filtro por rol), create, store, edit, update. Sin destroy.
6. CuentaRequest: name obligatorio; email obligatorio, formato y único (ignorando la cuenta que se edita); password obligatoria al crear (mínimo 8, confirmada) y opcional al editar; rol en la lista; supervisor_id obligatorio solo si rol = vendedor y debe existir como supervisor ACTIVO, y se guarda null para los demás roles; activo booleano. Reglas extra: el administrador no puede desactivarse ni cambiarse el rol a sí mismo; un supervisor con vendedores asignados no puede desactivarse ni cambiar de rol.
7. Menú según rol. Administrador: Cuentas, Tratamientos, Pacientes, Planes, Reporte. Coordinador: Mi equipo, Pacientes, Planes, Reporte. Después del login, /dashboard redirige a Pacientes.
8. Mensajes en español tratando de usted (en messages() del FormRequest). Los nombres de rol en pantalla: Administrador, Coordinador, Asesor.
```

✅ Comprueba (con `php artisan migrate:fresh --seed` recién corrido):

Prueba	**Resultado esperado**
Ir a `/register`	404
Login `valeria@crm.test`	No entra; mensaje de asesor
Login `ivan@crm.test`	No entra; mensaje de cuenta desactivada
Login `sergio@crm.test`	Entra; ve "Mi equipo", no ve "Cuentas"
Sergio va a `/cuentas` a mano	403
Login `admin@crm.test` → crear asesor sin coordinador	Error junto al campo
Admin intenta desactivarse	Error, no se guarda
Con Sergio logueado en otra ventana, admin lo desactiva	La siguiente petición de Sergio lo saca al login
Admin intenta desactivar a Sergio (tiene asesores)	Error

📝 `git commit -m "panel: cuentas y roles sin registro público"` → PR → B revisa.

> **En todos los specs que siguen:** cuando `/spec` guarde el archivo, apruébalo tú, haz commit del spec y ejecuta `/spec-impl NN-slug` con el nombre del spec (sección 0). Las pruebas ✅ de cada tarea son también sus criterios de aceptación.

6. Fase 2 · Catálogo, pacientes y equipo

Qué hace B al mismo tiempo: clientes y actividades de la API (A3, A5). Las dos implementan NIT y teléfono: al final de esta fase comparan resultados.

6.1 L3 · Catálogo de tratamientos

💻

```
git switch main && git pull && git switch -c spec-04-catalogo &&
claude
```

🤖

```
/spec Soy la integrante A. Spec 04-catalogo (requisito L3).
Objetivo: que el administrador mantenga el catálogo de tratamientos
sin borrarlos.

Decisiones cerradas:
- ProductoController solo para administrador: index (tabla con
nombre, precio en quetzales, requiere evaluación, estado), create,
store, edit, update y una acción para activar/desactivar. Sin
destroy.
- ProductoRequest: nombre obligatorio, máximo 120 y único (ignorando
el que se edita); precio obligatorio, numérico, mayor que 0, con
máximo 2 decimales (regla decimal:0,2) y máximo 99999999.99; activo y
requiere_evaluacion booleanos (casillas).
```

```
- Clase app/Support/Formato.php con quetzales($monto) → "Q1,200.00" y
fechaGT($fecha) → la fecha en America/Guatemala con formato d/m/Y
H:i. Úsala en todas las vistas.
- En pantalla se llama "Tratamientos". Cambiar un precio NO toca las
líneas de las oportunidades (no agregues observers ni nada que las
modifique).
```

✅

- Precio `0`, `-5` o `12.345` → error junto al campo, con lo escrito conservado.
- Cambia el precio de "Plan avanzado" y corre la consulta de Valeria de la sección 5.2: sigue dando 9250.00.

📝 `git commit -m "panel: catálogo de tratamientos sin borrado"`

6.2 L4 parte 1 · NIT, teléfono y pacientes

💻

```
git switch main && git pull && git switch -c spec-05-pacientes &&
claude
```

🤖

```
/spec Soy la integrante A. Spec 05-pacientes (requisito L4, parte 1;
la asignación va en el spec 07).
Objetivo: que administrador y coordinador busquen, creen y editen
pacientes con NIT y teléfono normalizados.

Decisiones cerradas:

1. app/Support/Nit.php:
   - normalizar(string $texto): string → quita espacios y guiones,
pasa a mayúsculas y quita ceros a la izquierda.
   - esConsumidorFinal(string $texto): bool → true si normalizado es
```

"CF".
   - esValido(string $compacto): bool → entre 2 y 12 caracteres, cuerpo solo dígitos, verificador 0-9 o K, módulo 11 como la sección 7 R4 del enunciado.
2. app/Support/Telefono.php:
   - normalizar(string $texto): string → quita espacios, guiones, puntos y paréntesis; quita "+502" si está al inicio; quita "502" del inicio solo si después quedan exactamente 8 dígitos.
   - esValido(string $normalizado): bool → /^[2-9][0-9]{7}$/.
3. Reglas app/Rules/NitValido.php (acepta CF o NIT válido) y TelefonoValido.php, con los mensajes:
   "Escriba un NIT válido (con su dígito verificador) o CF." y "Escriba un teléfono de Guatemala de 8 dígitos."
4. Pruebas unitarias tests/Unit/NitTest.php y TelefonoTest.php con TODOS los casos de la sección 10 de la especificación.
5. ClienteRequest: nombre obligatorio (máx 150); nit obligatorio (CF o NIT válido); telefono obligatorio y válido; correo opcional con formato (máx 150). Normaliza NIT y teléfono antes de comprobar que no se repitan (único ignorando el cliente que se edita; CF no se compara). Mensajes de repetido: "Ya existe un paciente con ese NIT." y "Ya existe un paciente con ese teléfono."
6. Scope Cliente::visiblePara(User $u): administrador ve todos; supervisor ve los de su equipo (vendedores con supervisor_id = él, activos o no) y los disponibles (vendedor_id null). Todas las acciones buscan con Cliente::visiblePara(auth()->user())->findOrFail($id), así lo ajeno da 404.
7. ClienteController (administrador y supervisor): index con búsqueda q (nombre o NIT, contiene), filtro por asesor (solo los visibles) y filtro "Sin asignar", paginado de 15, mostrando NIT como CF cuando es null; create/store (sin asignar todavía); edit/update; show con sus datos y planes. destroy solo para administrador y solo si no tiene oportunidades ("No se puede eliminar: el paciente tiene planes de tratamiento.").
8. Si al guardar MySQL responde error 1062 (duplicado que se coló entre la validación y el INSERT), vuelve al formulario con el error

```
junto al campo nit o telefono.
9. En pantalla se llama "Pacientes". Nunca {!! !!}.
```

✅ 💻

```
php artisan test --filter=Nit
php artisan test --filter=Telefono
```

Prueba	Resultado esperado
Crear paciente con NIT `452 1817-k`	"Ya existe un paciente con ese NIT" (es Comercial El Quetzal)
Teléfono `+502 5555-1234`	"Ya existe un paciente con ese teléfono"
Dos pacientes con NIT `CF`	Se guardan los dos
Sergio abre `/clientes/<id de Abarrotes San Juan>`	404 (es del equipo de Sandra)
Sergio ve la lista	Pacientes de Valeria, Diego y los 6 disponibles
Paciente llamado `<img src=x onerror=alert(1)>`	Se ve el texto tal cual, sin alerta

🔄 **Comparar con B:** pásale tu tabla de resultados de las pruebas unitarias; ella corre los mismos casos en la API. Si algún caso da distinto, es un defecto (L7).

📝 `git commit -m "panel: pacientes con NIT y teléfono normalizados"`

6.3 L4 parte 2 · Asignar, reasignar y liberar

💻

```
git switch main && git pull && git switch -c spec-07-asignacion &&
claude
```

🤖

/spec Soy la integrante A. Spec 07-asignacion (requisito L4, parte 2). Lee también las decisiones de la sección 5 de la especificación. Objetivo: asignar, reasignar y liberar pacientes respetando la cartera máxima aunque el panel y la API operen a la vez.

Decisiones cerradas:

Crea app/Services/Cartera.php con:
- asignar(Cliente $cliente, User $vendedor, User $actor): asigna un disponible o reasigna.
- liberar(Cliente $cliente, User $actor).

Dentro de DB::transaction, en este orden de candados: primero la fila del vendedor destino (User::whereKey(...)->lockForUpdate()->first()), luego la del cliente (lockForUpdate). Después:
- El vendedor debe ser rol vendedor y estar activo; si el actor es supervisor, el vendedor debe ser de su equipo y el cliente debe ser visible para él (si no, 404).
- Contar los clientes del vendedor; si ya tiene 10 → error "La cartera de <nombre> ya tiene 10 pacientes, el máximo permitido." (no cuenta si el cliente ya era suyo).
- Reasignar: cambiar clientes.vendedor_id y pasar al nuevo vendedor las oportunidades ABIERTAS de ese cliente; las cerradas se quedan con el vendedor anterior.
- Liberar: si el cliente tiene oportunidades abiertas → error "No se puede liberar: el paciente tiene planes de tratamiento abiertos."; si no, vendedor_id = null.
Los errores son una excepción propia (app/Exceptions/CarteraException) que el controlador muestra como mensaje en la página.

En ClienteController agrega asignar (PATCH clientes/{id}/asignar con vendedor_id, validado con AsignarRequest) y liberar (PATCH clientes/{id}/liberar). En la lista y el detalle: un select con los asesores activos que el usuario puede usar (con "x de 10" al lado de cada nombre) y un botón "Liberar".

```
El candado es el mismo que usa la API, para que el panel y la API no se pisen.
```

✅

Prueba	Resultado esperado
Sandra asigna un disponible a Carlos (tiene 9)	Se asigna; Carlos queda 10 de 10
Sandra asigna otro disponible a Carlos	"La cartera de Carlos Ajú ya tiene 10 pacientes…"
Sergio intenta asignar a Carlos (otro equipo)	No aparece en su lista; forzando la petición → 404
Sergio libera Comercial El Quetzal (tiene planes abiertos)	"No se puede liberar…"
Sergio reasigna Ferretería Xela de Valeria a Diego	Capacitación de personal (abierta) pasa a Diego; Equipo para sucursal (ganada) sigue de Valeria
Asignar a Iván (inactivo)	No aparece como opción

🔄 **Prueba con B:** asigna un paciente a Valeria en el panel y pídele a B que haga `GET /clientes` con el token de Valeria: el paciente debe aparecer. Es la primera mitad de la prueba central del proyecto.

📝 `git commit -m "panel: asignación de pacientes con candado de cartera"`

6.4 L5 · Mi equipo

💻

```
git switch main && git pull && git switch -c spec-09-equipo && claude
```

🤖

```
/spec Soy la integrante A. Spec 09-equipo (requisito L5).
Objetivo: que el coordinador vea la cartera y las ventas del mes de
cada asesor de su equipo.
Decisiones cerradas:
- EquipoController@index, solo supervisor: tabla con cada asesor de
su equipo (nombre, correo, estado, cartera "x de 10", ventas y total
vendido del mes).
- Selector de mes (?mes=AAAA-MM, validado con ^\d{4}-(0[1-9]|1[0-
2])$). Por omisión, el mes actual de Guatemala:
now('America/Guatemala')->format('Y-m').
- Las ventas del mes se calculan en la base con el constructor de
consultas: etapa = 'ganada' y DATE_FORMAT(fecha_cierre - INTERVAL 6
HOUR, '%Y-%m') = ? (el mes va como parámetro). Total = SUM(cantidad *
precio_unitario) de las líneas.
- Ruta /equipo con middleware rol:supervisor.
```

✅ Con la semilla de verificación y el mes `2026-10`, Sergio ve: Valeria 3 de 10, 2 ventas, Q9,250.00; Diego 2 de 10, 1 venta, Q6,000.00. Con `2026-11`: Valeria 1 venta, Q3,600.00.

📝 `git commit -m "panel: página de equipo con cartera y ventas del mes"`

---

7. Fase 3 · Base de la SPA, planes, reportes y semilla del sector

Qué hace B al mismo tiempo: oportunidades, regla propia y resumen en la API (A4, A6). **B necesita la base de la SPA para empezar el embudo en la fase 4**, así que empiezas esta fase por ella.

7.1 F6 + F1 · Base de la SPA y login

Primero crea la rama:

💻

```
git switch main && git pull && git switch -c spec-10-spa-base
```

Y **a mano** (tu agente no puede crearlos por la regla de `settings.local.json`), crea las dos vistas provisionales de B:

`frontend/js/vistas/embudo.js`

```
export default {
  titulo: 'Planes de tratamiento',
  async cargar(params) {},
  render(estado) { return '<p class="text-muted">Embudo en
construcción.</p>'; },
  acciones: {},
  formularios: {}
};
```

`frontend/js/vistas/plan.js`

```
export default {
  titulo: 'Detalle del plan',
  async cargar(params) {},
  render(estado) { return '<p class="text-muted">Detalle en
construcción.</p>'; },
  acciones: {},
  formularios: {}
};
```

Haz commit de esas dos vistas **antes** de `/spec-impl` (el skill pide que no haya cambios sin guardar):

💻

```
git add frontend/js/vistas && git commit -m "spa: vistas
provisionales del embudo y del plan"
```

```
claude
```

🤖

```
/spec Soy la integrante A. Spec 10-spa-base (requisitos F6 y F1).
Objetivo: tener la base de la SPA (estado, rutas, peticiones y
utilidades) y el inicio de sesión del asesor.
Decisiones cerradas: la arquitectura obligatoria está en la sección 3
de docs/plan.md ("Contrato de la SPA"): síguela al pie de la letra,
porque B programa embudo.js y plan.js contra ese contrato.

Crea en frontend/:
- index.html: Bootstrap 5.3 (CSS y bundle JS) desde CDN, <meta
viewport>, un <div id="app"></div> y <script type="module"
src="js/app.js">. Título "Clínica Dental Sonrisa · Asesores".
- css/estilos.css: lo mínimo (indicador de carga, columnas,
vencidas). Nada que provoque desplazamiento horizontal a 375 px.
- js/estado.js: el objeto estado de la sección 3.3, cambiar(parcial)
y suscribir(funcion).
- js/sesion.js: leerSesion(), guardarSesion(sesion), borrarSesion()
con localStorage, clave "sesion". Es lo único que se guarda en
localStorage.
- js/api.js: pedir(metodo, ruta, cuerpo) y ErrorApi según la sección
3.4. Base http://localhost:3000/api. Envía Content-Type JSON y
Authorization: Bearer. Suma y resta estado.cargando (en finally). Si
la respuesta es 401 y había sesión (y no es /auth/login):
borrarSesion(), aviso "Su sesión terminó. Ingrese de nuevo." y
location.hash = '#/login'. Si fetch falla (la API no responde):
ErrorApi con status 0 y el mensaje "No se pudo conectar con el
servidor. Verifique que la API esté encendida." Con 204 devuelve
null.
- js/util.js: escapar, quetzales (Intl es-GT, GTQ), fechaGT (Intl con
timeZone 'America/Guatemala', día, mes corto, año, hora y minutos),
aISO (agrega ':00-06:00' al valor de un datetime-local), mesActualGT,
errorCampo, claseError y mostrarError.
- js/textos.js: ETAPAS = { prospecto: 'Interesado', calificado:
```

```
'Evaluado', propuesta: 'Presupuesto enviado', negociacion: 'En
negociación', ganada: 'Aceptado', perdida: 'Rechazado' }, ABIERTAS,
CERRADAS, PROBABILIDAD, TIPOS = { llamada: 'Llamada', reunion:
'Cita', tarea: 'Tarea' }.
- js/app.js: rutas de la sección 3.1 (importa las 6 vistas, incluidas
embudo.js y plan.js que ya existen y NO debes modificar), lectura del
hash (#/plan/5 → ruta 'plan', params { id: '5' }), protección de
rutas sin sesión, layout (barra de navegación responsive de Bootstrap
con Pacientes, Planes, Actividades, Resumen, nombre del asesor y
"Salir"; indicador de carga cuando estado.cargando > 0; aviso que se
cierra con data-accion="cerrarAviso"), y la delegación: un solo
addEventListener('click') y uno de 'submit' sobre #app que buscan
[data-accion] y form[data-form] y llaman a vista.acciones[nombre]
(elemento, evento) o vista.formularios[nombre](datos, form) con
preventDefault. Acciones globales: cerrarAviso y salir. render() hace
#app.innerHTML = layout + vista.render(estado).
- js/vistas/login.js: formulario correo y clave (data-form="entrar");
POST /auth/login; con 200 guardarSesion y va a #/cartera; con 401
"Correo o contraseña incorrectos."; con 403 el mensaje que manda la
API; con status 0 el mensaje de la API apagada.
- js/vistas/cartera.js, actividades.js y resumen.js: provisionales
con la misma forma que embudo.js (se hacen en la fase 4).

Todo texto que venga de la API pasa por escapar(). Nada de
frameworks.
```

✅ Abre `frontend/` con Live Server (`http://127.0.0.1:5500`):

Prueba	**Resultado esperado**
API apagada → intentar entrar	Mensaje de que la API no responde (no pantalla vacía)
API encendida, `valeria@crm.test` / `Ventas2026`	Entra a Pacientes; se ve la barra
Login `sergio@crm.test`	Mensaje del 403 de la API
En la consola: `localStorage`	Solo existe la clave `sesion`

Prueba	Resultado esperado
Borra el token a mano (`localStorage.setItem('sesion', '{"token":"x"}')`) y navega	Vuelve al login con "Su sesión terminó…"
DevTools en 375 px	Sin barra de desplazamiento horizontal; el menú se colapsa
Click en "Planes"	Se ve "Embudo en construcción."

🔄 **Prueba de integración 2 (CORS):** si la petición falla con un error de CORS en la consola, el origen en el `.env` de la API de B no es exactamente `http://127.0.0.1:5500` . Avísale; **no lo cambies tú**.

📝

```
git add frontend && git commit -m "spa: base con estado, rutas,
delegación y login"
git push -u origin spec-10-spa-base
```

🔄 **Aviso a B:** "Ya está el PR `spec-10-spa-base` . Cuando lo unas, puedes empezar `embudo.js` y `plan.js` siguiendo la sección 3 de `plan.md` ."

7.2 L6 parte 1 · Planes de tratamiento (solo lectura)

💻

```
git switch main && git pull && git switch -c spec-12-planes && claude
```

🤖

```
/spec Soy la integrante A. Spec 12-planes (requisito L6, parte 1).
Objetivo: que administrador y coordinador consulten los planes de
tratamiento en solo lectura.
Decisiones cerradas:
- Scope Oportunidad::visiblePara(User $u): administrador todas;
```

```
supervisor las de los vendedores de su equipo. Lo ajeno → 404 con
findOrFail.
- OportunidadController@index: filtros etapa y asesor, columnas
paciente, título, asesor, etapa (nombre de pantalla), total, fecha de
cierre en hora de Guatemala. El total se calcula en la consulta con
una subconsulta SUM(cantidad * precio_unitario) (withSum no sirve
para multiplicar; usa selectSub o addSelect).
- OportunidadController@show: datos, líneas (tratamiento, cantidad,
precio guardado, subtotal), total e historial (etapa anterior →
nueva, quién y cuándo en hora de Guatemala).
- Sin botones de editar: los planes los trabaja el asesor en la SPA.
En pantalla se llaman "Planes de tratamiento".
```

✅ Como admin: "Renovación de planes" muestra total Q5,650.00, Plan avanzado a Q2,600.00 y 5 registros en el historial. Sergio no puede abrir "Plan anual" (de Carlos) → 404.

📝 `git commit -m "panel: planes de tratamiento de solo lectura"`

7.3 L6 parte 2 · Reporte del mes

💻

```
git switch main && git pull && git switch -c spec-14-reporte &&
claude
```

🤖

```
/spec Soy la integrante A. Spec 14-reporte (requisito L6, parte 2).
Objetivo: mostrar el reporte del mes por asesor, tratamiento y
equipo, calculado en la base.
Decisiones cerradas: reglas R7 del enunciado y la sección 11 de la
especificación.

- ReporteController@index (administrador y supervisor) con ?mes=AAAA-
```

```
MM (validar con regex ^\d{4}-(0[1-9]|1[0-2])$; por omisión el mes
actual de Guatemala), y un <input type="month"> para elegirlo.
- Todo se calcula en MySQL con el constructor de consultas. El mes va
siempre como parámetro: whereRaw("DATE_FORMAT(o.fecha_cierre -
INTERVAL 6 HOUR, '%Y-%m') = ?", [$mes]).
- Tabla por asesor: ventas (COUNT DISTINCT de oportunidades ganadas
del mes), total vendido (SUM cantidad × precio_unitario), pronóstico
(ROUND(SUM(cantidad × precio_unitario × probabilidad), 2) de las
abiertas SIN filtro de mes, con CASE etapa WHEN 'prospecto' THEN 0.10
... como decimales) y cartera "x de 10". Incluye asesores sin ventas.
- Tabla por tratamiento: unidades vendidas (SUM cantidad) y total, de
las ventas del mes.
- Fila de totales al final. Para el administrador, además, una tabla
por equipo (coordinador) con ventas, total y pronóstico.
- El pronóstico de un grupo se redondea al final (ROUND de la suma),
no sumando valores ya redondeados.
- El supervisor ve solo su equipo; el administrador, a todos. Nada se
calcula en PHP con float.
```

✅ Corre `php artisan migrate:fresh --seeder=VerificacionSeeder` y compara **cada número** con la sección 11 de la especificación:

Quién	Mes	Ventas	Total	Pronóstico	Cartera
Valeria (admin o Sergio)	2026-10	2	Q9,250.00	Q2,512.88	3 de 10
Valeria	2026-11	1	Q3,600.00	Q2,512.88	3 de 10
Diego	2026-10	1	Q6,000.00	Q112.50	2 de 10
Carlos (admin o Sandra)	2026-10	1	Q7,200.00	Q4,125.75	9 de 10
Equipo de Sergio	2026-10	3	Q15,250.00	Q2,625.38	
Equipo de Sandra	2026-10	1	Q7,200.00	Q4,125.75	
Toda la empresa	2026-10	4	Q22,450.00	Q6,751.13	

Por tratamiento, octubre: Plan básico 5 / Q6,000.00 · Plan avanzado 2 / Q5,200.00 · Instalación 1 / Q450.00 · Soporte anual 3 / Q10,800.00.

> Si a Valeria le salen Q5,650.00 → estás agrupando en UTC. Si salen Q9,551.00 → estás usando el precio del catálogo. Cualquiera de los dos es un buen caso para la bitácora.

Toma captura del reporte de octubre (como admin) para `verificacion.md` .

📝 `git commit -m "panel: reporte del mes por asesor, tratamiento y equipo"`

7.4 L7 · Validación y mensajes en español

💻

```
git switch main && git pull && git switch -c spec-16-validacion &&
claude
```

🤖

```
/spec Soy la integrante A. Spec 16-validacion (requisito L7).
Objetivo: que todo el panel valide en el servidor con mensajes en
español que tratan de usted.
Decisiones cerradas:
- Ejecuta php artisan lang:publish y crea lang/es/ (auth.php,
validation.php, passwords.php, pagination.php) traducido al español
tratando de usted, con los nombres de atributos en español (name →
nombre, email → correo, password → contraseña, nit → NIT, telefono →
teléfono, precio → precio, supervisor_id → coordinador).
- Revisa TODOS los formularios del panel: cada campo muestra @error
junto a él y conserva old(). Ningún {!! !!} en resources/views
(búscalos con grep y dime si hay).
- Confirma que NitValido, TelefonoValido y las mismas reglas de
cartera se usan en todos los lugares donde se crea o edita un
cliente.
```

✅ Envía cada formulario vacío: todos los mensajes salen en español y tratan de usted. `grep -rn "{!!" laravel/resources/views` no devuelve datos del usuario.

📝 `git commit -m "panel: mensajes de validación en español"`

7.5 L1 · Semilla del sector

💻

```
git switch main && git pull && git switch -c spec-17-semilla-sector
&& claude
```

🤖

```
/spec Soy la integrante A. Spec 17-semilla-sector (requisito L1, capa
de sector).
Objetivo: cargar datos creíbles de la clínica que permitan demostrar
la regla propia.
Decisiones cerradas: llenar SectorSeeder (corre después de
VerificacionSeeder). Reglas: no repetir correos, NIT ni teléfonos de
la semilla de verificación; máximo 10 clientes por vendedor;
contraseña Hash::make('Ventas2026') (es la única permitida en el
repo); historial con la misma lógica que VerificacionSeeder (3 días
entre registros, terminando en la fecha de cierre o en 2026-11-02
16:00:00 UTC si sigue abierta); todas las fechas en UTC.

Cuentas: Dra. Lucía Méndez (lucia@dental.test, supervisor); Andrea
Gómez (andrea@dental.test) y Kevin Chávez (kevin@dental.test),
vendedores de Lucía.

Tratamientos (nombre, precio, requiere_evaluacion): Limpieza dental
300.00 no · Resina dental 350.00 no · Extracción simple 250.00 no ·
Extracción de cordal 900.00 no · Blanqueamiento 1200.00 no ·
Endodoncia 1500.00 no · Corona de porcelana 2500.00 no · Ortodoncia
con brackets 8000.00 sí · Implante dental 9000.00 sí.

Pacientes (nombre, NIT o CF, teléfono, correo opcional):
Andrea: Sofía Ramírez CF 48101101 sofia.ramirez@correo.test · Jorge
Cifuentes 10293841 48101102 · Colegio Bilingüe Los Altos 56473826
```

77681103 administracion@colegiolosaltos.test · Mariela Chun CF 48101104 · Rodrigo Tzoc 91827361 48101105 · Gabriela Barrios CF 48101106
Kevin: Esteban Quemé 33445567 51201201 · Paola Fernández CF 51201202 · Textiles del Altiplano 70123454 77681204 · Héctor Sapón 24681350 51201205 · Claudia Ixcot CF 51201206
Disponibles: Fernando Pú 81357923 51301301 · Daniela Monterroso CF 51301302 · Andrés Velásquez 60708093 51301303

Planes (título, paciente, vendedor, etapa, cierre UTC, líneas cantidad × precio):
- Ortodoncia completa · Sofía Ramírez · Andrea · calificado · — · Ortodoncia con brackets 1 × 8000.00
- Implante de molar · Jorge Cifuentes · Andrea · negociacion · — · Implante dental 1 × 9000.00
- Limpieza para docentes · Colegio Bilingüe Los Altos · Andrea · ganada · 2026-10-20 16:00 · Limpieza dental 25 × 300.00
- Blanqueamiento · Mariela Chun · Andrea · propuesta · — · Blanqueamiento 1 × 1200.00
- Endodoncia y corona · Rodrigo Tzoc · Andrea · perdida desde negociacion, motivo "Decidió tratarse en otra clínica" · 2026-10-28 18:00 · Endodoncia 1 × 1500.00; Corona de porcelana 1 × 2500.00
- Primera consulta · Gabriela Barrios · Andrea · prospecto · — · sin líneas
- Resinas · Esteban Quemé · Kevin · ganada · 2026-11-03 17:00 · Resina dental 3 × 350.00
- Extracción de cordales · Paola Fernández · Kevin · negociacion · — · Extracción de cordal 2 × 900.00
- Limpieza anual · Textiles del Altiplano · Kevin · calificado · — · Limpieza dental 12 × 300.00
- Implante dental · Héctor Sapón · Kevin · calificado · — · Implante dental 1 × 9000.00
- Ortodoncia · Claudia Ixcot · Kevin · prospecto · — · Ortodoncia con brackets 1 × 8000.00

Actividades (paciente, vendedor, tipo, descripción, vence UTC, completada):

```
- Jorge Cifuentes · Andrea · reunion · Cita de evaluación para
implante · 2026-10-10 15:00 · sí
- Héctor Sapón · Kevin · reunion · Cita de evaluación con radiografía
· 2026-10-30 15:00 · sí
- Sofía Ramírez · Andrea · reunion · Cita de evaluación de ortodoncia
· 2026-11-20 15:00 · no
- Mariela Chun · Andrea · llamada · Confirmar presupuesto de
blanqueamiento · 2026-11-25 15:00 · no
- Paola Fernández · Kevin · tarea · Enviar presupuesto de cordales ·
2026-11-01 15:00 · no
- Colegio Bilingüe Los Altos · Andrea · llamada · Coordinar horarios
de limpieza · 2026-10-18 15:00 · sí
- Claudia Ixcot · Kevin · reunion · Agendar evaluación de ortodoncia
· 2026-11-28 15:00 · no

Con estos datos se demuestra la regla propia: Sofía (ortodoncia, cita
pendiente) no puede pasar a propuesta; Héctor (implante, cita
completada) sí.
```

✅ 💻

```
php artisan migrate:fresh --seed
php artisan migrate:fresh --seeder=VerificacionSeeder   # y repite
las consultas de la sección 5.2: deben dar igual
```

Entra como `lucia@dental.test` y revisa "Mi equipo", Pacientes y el reporte.

> Los precios son aproximados; si puedes, confírmalos con alguna clínica o página de Quetzaltenango antes de entregar.

📝 `git commit -m "panel: semilla del sector dental"`

🔄 **Aviso a B:** "Ya está la semilla del sector. Sofía Ramírez y Héctor Sapón sirven para la prueba de la regla propia en Postman."

---

8. Fase 4 · Vistas de la SPA

Qué hace B al mismo tiempo: `embudo.js` y `plan.js`, y la colección de Postman. Ustedes dos editan archivos distintos de `frontend/js/vistas/`, así que no hay choque. Si ella necesita una función nueva en `util.js` o `textos.js`, te la pide y tú la agregas en un PR pequeño.

8.1 F2 · Cartera

💻

```
git switch main && git pull && git switch -c spec-18-spa-cartera &&
claude
```

🤖

```
/spec Soy la integrante A. Spec 18-spa-cartera (requisito F2).
Objetivo: que el asesor gestione su cartera de pacientes desde la
SPA.
Decisiones cerradas: usa el contrato de la sección 3 de docs/plan.md
y los endpoints de la sección 4 de la especificación. Solo modificas
js/vistas/cartera.js (y, si hace falta, util.js o textos.js).

- cargar(): en paralelo GET /clientes (o /clientes?q= si hay
búsqueda), GET /clientes/disponibles y GET /reportes/resumen (para
cartera.clientes y cartera.maximo). Guarda en estado.datos.cartera.
- render(): título "Mis pacientes" con "x de 10"; formulario de
búsqueda (data-form="buscar", la búsqueda la hace el servidor); tabla
responsive con nombre, NIT, teléfono y correo; formulario "Nuevo
paciente" (data-form="nuevoPaciente") con nombre, NIT ("escriba CF si
es consumidor final"), teléfono y correo opcional; lista "Pacientes
disponibles" (solo id y nombre) con botón data-accion="tomar" data-
id.
- nuevoPaciente: POST /clientes. Con 201 limpia el formulario, aviso
de éxito y recarga. Con 422 guarda { valores, errores } en
```

```
estado.formularios.nuevoPaciente: los errores salen junto a cada
campo y lo escrito se conserva. Con 409 aviso con el mensaje de la
API.
- tomar: POST /clientes/:id/tomar. Con 200 aviso "Paciente agregado a
su cartera" y recarga. Con 409 aviso con el mensaje de la API (otro
asesor lo tomó o la cartera está llena).
- Todo con escapar().
```

✅

Prueba	**Resultado esperado**
Valeria entra	3 pacientes, "3 de 10", 6 disponibles
Buscar `quetzal`	Solo Comercial El Quetzal
Nuevo paciente con NIT `4521817-1`	Error del NIT junto al campo; los demás campos conservan lo escrito
Tomar Café Zunil	Pasa a la cartera; "4 de 10"
Dos ventanas (Valeria y Diego) toman el mismo disponible	Uno lo consigue, el otro ve el mensaje del 409
Paciente `<img src=x onerror=alert(1)>`	Se ve como texto

📝 `git commit -m "spa: cartera con búsqueda, alta y tomar disponibles"`

8.2 F4 · Actividades

💻

```
git switch main && git pull && git switch -c spec-20-spa-actividades
&& claude
```

🤖

```
/spec Soy la integrante A. Spec 20-spa-actividades (requisito F4).
Objetivo: que el asesor vea, cree y complete sus actividades con
```

```
fechas en hora de Guatemala.
Decisiones cerradas: solo modificas js/vistas/actividades.js.

- cargar(): GET /actividades?estado=pendiente o completada (según la
pestaña guardada en estado.datos.actividades.pestana, por omisión
pendiente) y GET /clientes (para el select de pacientes).
- render(): pestañas Pendientes / Completadas (data-accion="pestana"
data-valor); lista con paciente, tipo con su nombre de pantalla (Cita
en lugar de reunion), descripción, fecha de vencimiento con
fechaGT(); las que tienen vencida = true se ven en rojo con la
etiqueta "Vencida"; botón data-accion="completar" data-id en las
pendientes.
- Formulario "Nueva actividad" (data-form="nuevaActividad"): paciente
(select), tipo, descripción, fecha y hora (input datetime-local,
opcional). POST /clientes/:id/actividades con vence_en = aISO(valor)
solo si se escribió. Con 422, errores junto a cada campo y lo escrito
conservado.
- completar: PATCH /actividades/:id con { completada: true } y
recarga.
```

✅

- Valeria ve "Presentar opciones de soporte" (vence 1 de septiembre) en rojo como **Vencida**.
- Crea una cita para el 1 de diciembre a las 10:00: en la API se guarda `2026-12-01T16:00:00.000Z` y en la SPA se ve a las 10:00.
- Cambia la zona horaria de tu computadora (o usa DevTools → Sensors → Location) y comprueba que las horas que muestra no cambian.

📝 `git commit -m "spa: actividades pendientes, completadas y vencidas"`

8.3 F5 · Resumen del mes

💻

```
git switch main && git pull && git switch -c spec-22-spa-resumen &&
claude
```

🤖

```
/spec Soy la integrante A. Spec 22-spa-resumen (requisito F5).
Objetivo: que el asesor consulte su resumen de cualquier mes.
Decisiones cerradas: solo modificas js/vistas/resumen.js.
- Input type="month" (por omisión mesActualGT()) dentro de un form
data-form="mes".
- GET /reportes/resumen?mes=AAAA-MM. Cuatro tarjetas Bootstrap:
"Planes aceptados" (ventas), "Total vendido" (quetzales),
"Pronóstico" (quetzales, con una nota de que no depende del mes) y
"Cartera" (x de 10).
- Si el mes es inválido y la API responde 400, aviso con su mensaje.
```

✅ Valeria, octubre 2026: 2 · Q9,250.00 · Q2,512.88 · 3 de 10. Noviembre 2026: 1 · Q3,600.00 · Q2,512.88.

📝 `git commit -m "spa: resumen del mes con selector"`

8.4 🔄 Prueba central del proyecto (con B)

Cuando el embudo de B esté unido:

1. `php artisan migrate:fresh --seeder=VerificacionSeeder` .
2. **Panel → SPA:** Sergio asigna "Escuela Futuro" a Valeria en el panel → Valeria la ve en su cartera de la SPA.
3. **SPA → Panel:** Valeria crea un plan para Escuela Futuro, le agrega un producto y lo lleva hasta "Aceptado" en la SPA → el reporte del mes actual del panel cuenta esa venta.
4. Anota los dos resultados con capturas en `verificacion.md` .

---

9. Fase 5 · Verificación y entrega

Qué hace B al mismo tiempo: el esqueleto de `verificacion.md`, las pruebas simultáneas de la API, el README y la colección de Postman.

9.1 Tu parte de `verificacion.md`

Espera a que B suba el esqueleto (🔄). Luego, una rama por bloque:

💻

```
git switch main && git pull && git switch -c docs/verificacion-a && claude
```

🤖 (uno por requisito: L1, L2, L3, L4, L5, L6, L7, F1, F2, F4, F5, F6)

```
/verifica L4
```

Copia las filas que te da en tu subsección de `verificacion.md`. Las pruebas que te tocan en la sección de seguridad:

Prueba	Evidencia
S1	`SELECT LEFT(password,4) FROM users` → solo `$2y$`; `git grep -n "Ventas2026"` solo aparece en los seeders, el README y la colección de Postman
S2	`git log --all -- '*.env'` vacío; `.env.example` sin valores reales
S3	Buscar `' OR 1=1 --` en Pacientes: no rompe nada ni muestra todo
S5	Sergio abre a mano un paciente de Sandra → 404 (captura)
S7	Paciente `<img src=x onerror=alert(1)>` en el panel y en la SPA, sin alerta (captura)
Cifras	Reporte de octubre del panel coincide con la tabla de control (captura)

9.2 Bitácora de IA

Durante todo el proyecto, cada vez que el agente proponga algo incorrecto (sección 11 de esta guía), anótalo en el momento con:

🤖

```
/bitacora A
```

Necesitas **al menos 2 casos tuyos**, cada uno con el hash del commit que lo corrigió (`git log --oneline`).

9.3 Pruebas automáticas (reto opcional, suma nota)

Ya tienes `NitTest` y `TelefonoTest` . Si te sobra tiempo:

🤖

```
Soy la integrante A. Reto opcional: pruebas del panel. Crea
tests/Feature con pruebas que usen la base de pruebas (configura
phpunit.xml para MySQL con una base crm_dental_test, NO la de
desarrollo) y corran VerificacionSeeder:
1. Un vendedor no puede iniciar sesión en el panel.
2. Sergio recibe 404 al abrir un cliente del equipo de Sandra.
3. Asignar a Carlos cuando tiene 10 clientes falla con el mensaje de
cartera llena.
4. El reporte de octubre 2026 de la empresa da 4 ventas, 22450.00 y
6751.13.
No uses SQLite: las CHECK y DATE_FORMAT son de MySQL.
```

9.4 Actualizar `plan.md` y cerrar

- Marca el estado de cada tarea en la sección 4 de `plan.md` y agrega la línea final en el historial de cambios.

- Revisa con B el README (lo escribe ella): los pasos para levantar el panel deben ser los tuyos. Pásale este bloque:

```
cd laravel
composer install
cp .env.example .env        # poner usuario y contraseña de MySQL
php artisan key:generate
npm install && npm run build
php artisan migrate:fresh --seed
php artisan serve           # http://localhost:8000
```

9.5 🔄 Etiqueta `entrega-final` (juntas)

Con todo unido a `main` y probado desde cero en una computadora limpia:

💻

```
git switch main && git pull
git tag -a entrega-final -m "Entrega final"
git push origin entrega-final
git rev-parse entrega-final   # este hash se envía en el aula virtual
```

---

10. Cambio de requisito (spec-v2)

Cuando el catedrático publique el cambio:

1. **Juntas**, decidan qué partes afecta y quién es la dueña de cada una.
2. Actualiza tus secciones de la especificación en una rama `docs/spec-v2` :

🤖

```
Soy la integrante A. El catedrático publicó este cambio de requisito:
<pega el texto>.
```

```
Sin escribir código: dime qué secciones de docs/especificacion.md
cambian (solo las mías: 1, 2, 3, 6, F1, F2, F4, F5, F6, 9 y 11), qué
migraciones NUEVAS harían falta (las existentes no se editan), qué
criterios Dado/Cuando/Entonces hay que agregar o cambiar y qué partes
de la API o del embudo (de B) se verían afectadas para avisarle.
Después redacta los cambios en mis secciones.
```

3. Unan los dos PR de especificación y etiqueten `spec-v2` (`git tag -a spec-v2 -m "..." && git push origin spec-v2`).
4. Agrega la fase del cambio en `plan.md` .
5. Implementa con un spec nuevo: anota su número (desde el 24) en `specs/README.md` , crea la rama `spec-NN-slug` , usa `/spec` y `/spec-impl` , **siempre con migraciones nuevas**. Los specs que el cambio deja viejos pasan a **Obsoleto** (no se borran).
6. Vuelve a correr `migrate:fresh --seeder=VerificacionSeeder` y verifica que las cifras de control no cambiaron.

---

11. Errores típicos del agente

Vigila estos errores en cada plan que te proponga Claude Code. Cuando atrapes uno, **es un caso para la bitácora**.

Error del agente	Por qué está mal	Cómo lo notas
Edita `create_users_table` para agregar `rol`	L1 exige una migración nueva	El plan menciona ese archivo
Pone `->nullOnDelete()` en `supervisor_id`	MySQL rechaza la CHECK sobre esa columna	`migrate` falla con error 3823
Usa `timestamp` para `fecha_cierre`	Puede convertir la hora según la zona de la sesión	Revisa el tipo en la migración
Calcula el reporte con `->get()` y suma en PHP	R7 pide calcular en la base, y PHP usa float	Hay `sum()` o `foreach` sumando en el controlado
Agrupa el mes con `MONTH(fecha_cierre)`	Agrupa en UTC: Valeria sale Q5,650.00	Cifras de control

Error del agente	Por qué está mal	Cómo lo notas
Usa el precio del catálogo en el total	R6: el precio va guardado en la línea	Valeria sale Q9,551.00
Redondea el pronóstico de cada plan y luego suma	Las cifras de control redondean la suma	Equipo o empresa con 1 centavo de diferencia
Deja la ruta `/register` o "Eliminar cuenta"	L2: no hay registro y las cuentas no se borran	Abre `/register` y el perf
Responde 403 a un paciente de otro equipo	R2/L5: debe ser 404	Prueba de Sergio
Usa `{!! $cliente->nombre !!}`	S7	Prueba del `<img onerror`
Asigna sin `lockForUpdate`	S8: dos asignaciones simultáneas pasan la cartera	El plan no menciona candado
Valida NIT con una regex sin el módulo 11	R4	`4521817-1` pasa
Quita siempre el `502` del teléfono	`5021 2345` queda inválido	Prueba unitaria
Guarda en localStorage algo más que la sesión	F1	Consola → `localStorage`
Usa `textContent` / `innerHTML` sin escapar en una vista	S7	Prueba del `<img onerror` en la SPA
Pone una contraseña distinta de `Ventas2026` en la semilla del sector	S1	`grep` de contraseñas
Modifica `embudo.js` o algo de `api/`	Archivos de B	Claude Code lo bloquea (l verás como error de permisos)

---

12. Preparación de la defensa

La nota es **individual** y tienes que poder responder sobre **cualquier** parte, incluida la de B. Te pueden pedir cambios en vivo.

12.1 Lo tuyo: preguntas probables

Pregunta	Idea de la respuesta
¿Por qué solo Laravel crea tablas?	Dos aplicaciones escriben en la misma base; si las dos cambian el esquema, se desincronizan. Un solo dueño = una sola versión del esquema, versionada en migraciones
¿Por qué solo el admin crea cuentas?	Un solo lugar genera hashes (S1) y controla roles; no hay registro público por el que alguien se dé rol
¿Para qué las CHECK si el panel ya valida?	La API también escribe; una regla que vive solo en una app no protege a la otra
¿Por qué `supervisor_id` no tiene `ON DELETE` ?	MySQL prohíbe acciones referenciales en columnas de una CHECK; y las cuentas no se borran
¿Cómo evitas que el coordinador vea otro equipo?	`Cliente::visiblePara()` + `findOrFail` → 404, no 403, para no confirmar que existe
¿Cómo se respeta la cartera de 10 si el panel y la API asignan a la vez?	Transacción + `FOR UPDATE` en la fila del vendedor, en el mismo orden en las dos puertas
¿Por qué Valeria tiene Q9,250 y no Q5,650 en octubre?	La venta de 03:30 UTC del 1 de noviembre es el 31 de octubre a las 21:30 en Guatemala; agrupamos con `fecha_cierre - INTERVAL 6 HOUR`
¿Por qué el precio se guarda en la línea?	El precio cotizado no cambia si después cambia el catálogo
¿Cómo funciona la SPA sin framework?	Un estado único, `cambiar()` vuelve a dibujar, y un solo escuchador por tipo de evento (delegación)
¿Por qué escapar si los datos vienen de nuestra API?	El token está en localStorage; un nombre con `<script>` lo podría robar

12.2 Lo de B: lo mínimo que debes saber explicar

Usa `/revisa` sobre cada PR de B y `/explica api/src` antes de la defensa.

Asegúrate de entender:

- Cómo `POST /clientes/:id/tomar` es atómico (`UPDATE ... WHERE vendedor_id IS NULL` + candado del vendedor).
- Por qué el middleware vuelve a leer al usuario en cada petición (para que un token de una cuenta desactivada deje de servir).
- Cómo el cambio de etapa guarda el historial en la misma transacción y por qué dos cambios simultáneos no dejan dos registros (`FOR UPDATE` + índice único).
- Por qué `bcryptjs` y no `bcrypt` (el prefijo `$2y$` de PHP).
- Por qué `decimalNumbers: true` en mysql2.
- Dónde se cumple la regla propia en la API.

12.3 Cambios en vivo para practicar

Practica cada uno sin agente, con un cronómetro de 5 minutos:

1. Cambiar la cartera máxima a 12 (¿en cuántos lugares está el 10? deberían ser pocos: busca una constante).
2. Agregar un filtro "solo activos" a la lista de cuentas.
3. Agregar la columna "correo" a la tabla de "Mi equipo".
4. Cambiar el texto "Presupuesto enviado" por "Presupuesto entregado" en el panel y en la SPA (`textos.js`).
5. Agregar una columna nueva a `productos` (con migración nueva, que acepte vacío).
6. Mostrar en el reporte el número de planes perdidos del mes.

12.4 Ensayo cruzado (una semana antes)

Cada una le hace a la otra 5 preguntas sobre **su** parte y le pide un cambio en vivo. Luego cambian de papel.

12.5 El día de la defensa

💻

```
git switch main && git checkout entrega-final
docker start mysql-crm
cd laravel && php artisan migrate:fresh --seeder=VerificacionSeeder
&& php artisan serve
# en otra terminal: la API de B (cd api && npm run dev)
# Live Server sobre frontend/, Postman abierto
```

---

13. Apéndices

13.1 Mensajes de sincronización (copia y pega)

Momento	Mensaje a B
Repo creado	"Ya está el repo. Clona, copia tu enunciado a `docs/` y crea tu `settings.local.json`."
Spec de A lista	"Subí mis secciones en `docs/spec-a`, ¿la revisas? Yo reviso la tuya."
`spec-v1`	"Ya está la etiqueta spec-v1. A programar."
Esquema listo	"PR `spec-01-esquema-y-semilla` listo. Después de unirlo: `git pull`, `composer install`, `.env`, `php artisan migrate:fresh --seed`."
NIT y teléfono	"Te paso mis resultados de la tabla de casos; ¿te dan igual en la API?"
Asignación lista	"Asigné un paciente a Valeria en el panel, ¿lo ves en `GET /clientes`?"
Base de la SPA	"PR `spec-10-spa-base` listo. Ya puedes hacer `embudo.js` y `plan.js` (sección 3 de `plan.md`)."
Semilla del sector	"Semilla del sector lista: Sofía (sin cita) y Héctor (con cita) para la regla propia."
Reporte listo	"Mi reporte de octubre da las cifras de control; ¿tu resumen también?"
Necesito algo de la API	"En `<endpoint>` necesito `<cambio>`. ¿Lo puedes agregar?"

13.2 Ramas de A, en orden

```
docs/spec-a → panel/entorno → docs/plan → (spec-v1)
spec-01-esquema-y-semilla → spec-03-cuentas-roles
spec-04-catalogo → spec-05-pacientes → spec-07-asignacion → spec-09-
equipo
spec-10-spa-base → spec-12-planes → spec-14-reporte → spec-16-
validacion → spec-17-semilla-sector
spec-18-spa-cartera → spec-20-spa-actividades → spec-22-spa-resumen
docs/verificacion-a → (entrega-final)
```

Los specs de B usan los números que faltan (02, 06, 08, 11, 13, 15, 19, 21 y 23).

Unir cada PR con **"Create a merge commit"** (no "Squash"): así el historial conserva tus commits repartidos en el tiempo, que también se evalúa.

13.3 Comandos de Claude Code que más vas a usar

Comando o tecla	Para qué
`Shift+Tab` (dos veces)	Modo plan: el agente propone antes de tocar archivos
`Esc`	Detener al agente si se va por otro lado
`/clear`	Limpiar el contexto entre tareas
`/permissions`	Ver qué tiene permitido y prohibido
`@ruta/archivo`	Señalar un archivo concreto en tu mensaje
`/spec` , `/spec-impl NN-slug`	Diseñar e implementar cada feature (fernando-skills)
`/seccion` , `/verifica` , `/explica` , `/revisa` , `/bitacora`	Los comandos propios del proyecto (sección 3.4)